Transformers Learn Generalizable CoT Reasoning via Gradient Descent

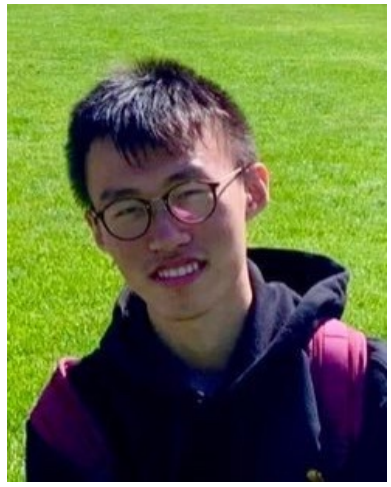
Yuejie Chi

Yale Statistics & Data Science

MIT IDSS Seminar
2025

Tong Yang

CMU



Zixin Wen

CMU

Yu Huang

Penn

Yingbin Liang

OSU

Yuxin Chen

Penn

Aarti Singh

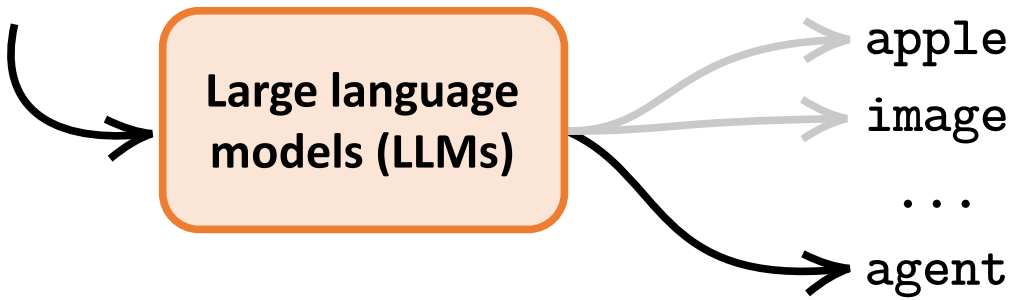
CMU

Large language models

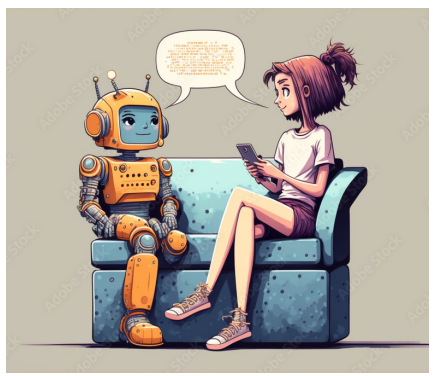
The language models predict the next word based on previous words.

Prompt: Explain reinforcement learning (RL).

Answer: Reinforcement learning (RL) is a type of machine learning where an...



Applications in translation, Q&A, summarization...

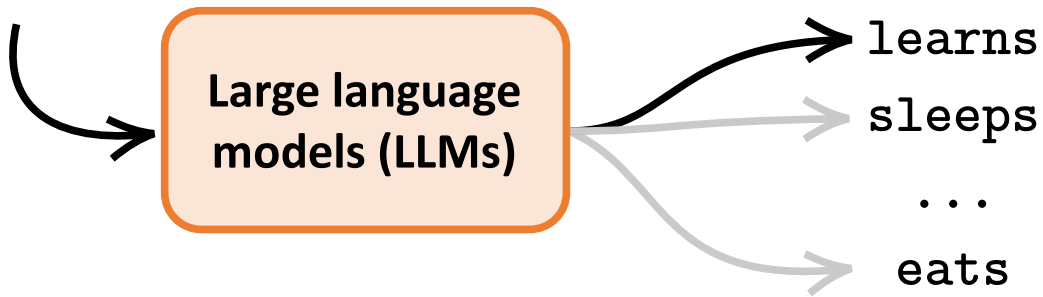


Large language models

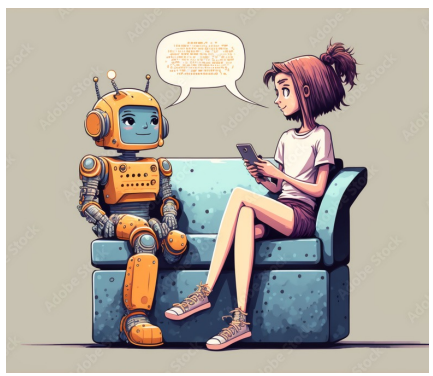
The language models predict the next word based on previous words.

Prompt: Explain reinforcement learning (RL).

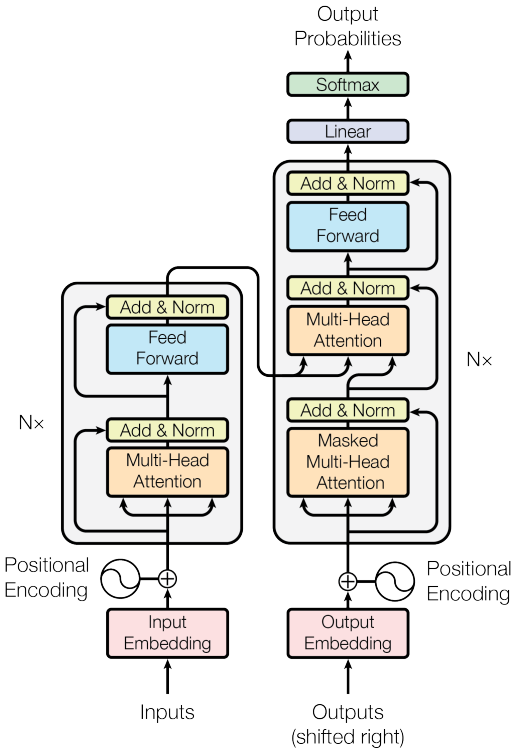
Answer: Reinforcement learning (RL) is a type of machine learning where an agent



Applications in translation, Q&A, summarization...



Transformers as the backbone architecture



Attention is all you need

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

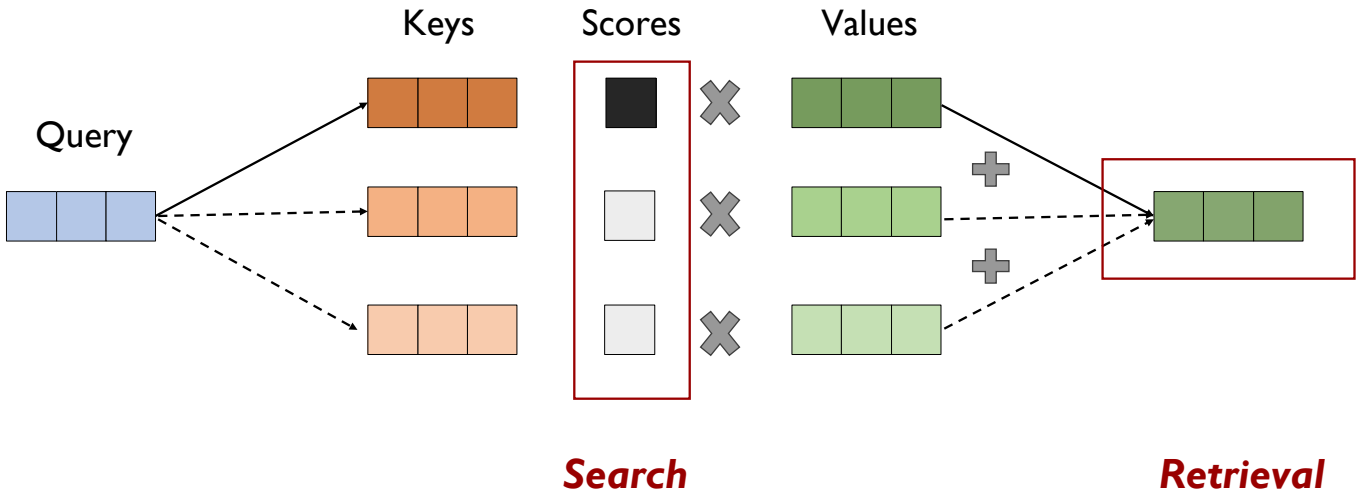
The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configuration. The best …

☆ Save 99 Cite Cited by 188409 Related articles »

Attention head

Attention head: For each query q, it matches against all the key-value pairs (k_i, v_i) and compute the value v:

$$\text{head}(q) = \sum \alpha_i v_i, \quad \text{where} \quad \alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_u \exp(q^\top k_u/\sqrt{d})}.$$

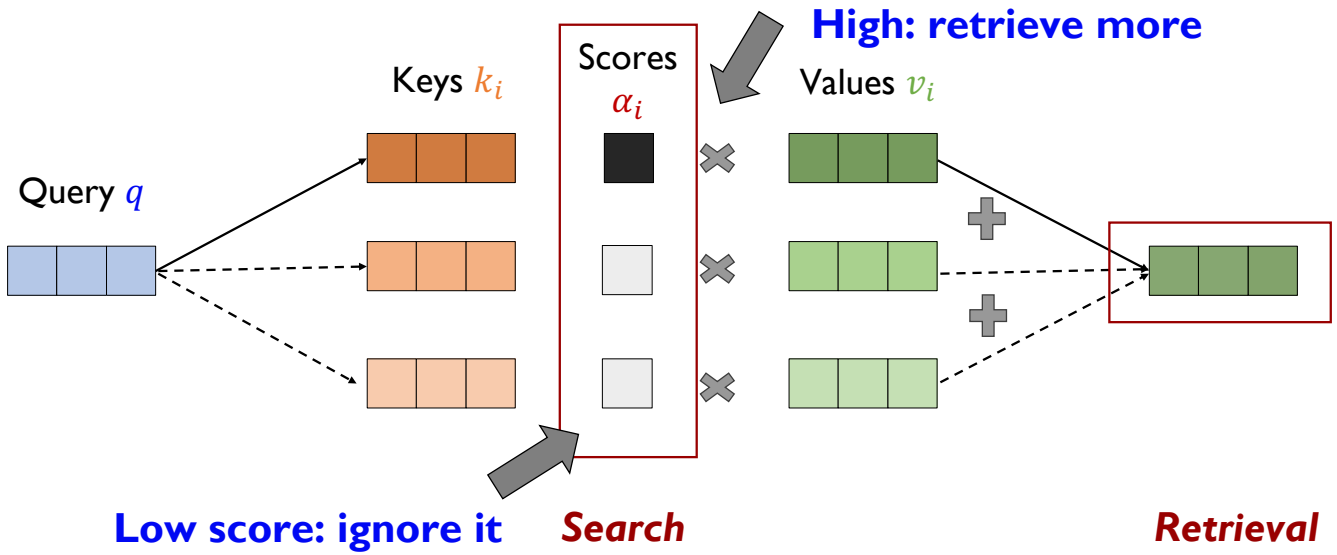


Attention searches and retrieves the values of relevant tokens

Attention head

Attention head: For each query q, it matches against all the key-value pairs (k_i, v_i) and compute the value v:

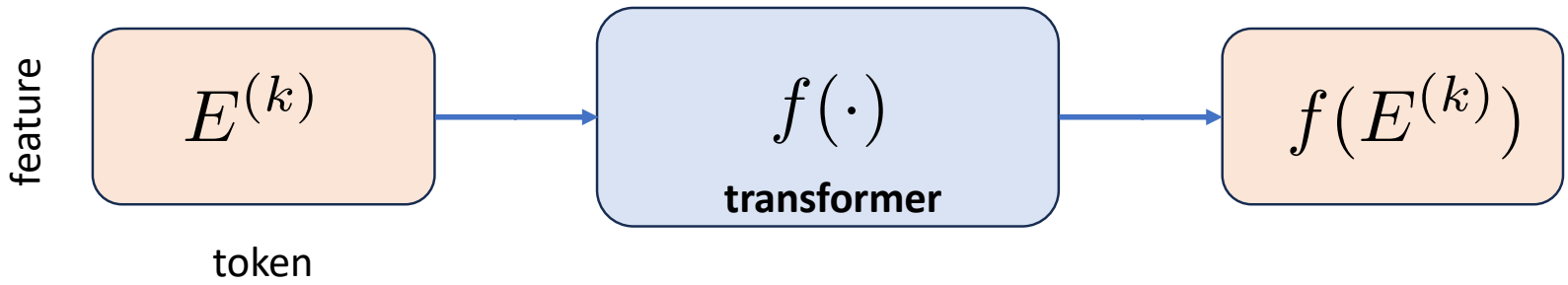
$$\text{head}(q) = \sum \alpha_i v_i, \quad \text{where} \quad \alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_u \exp(q^\top k_u/\sqrt{d})}.$$



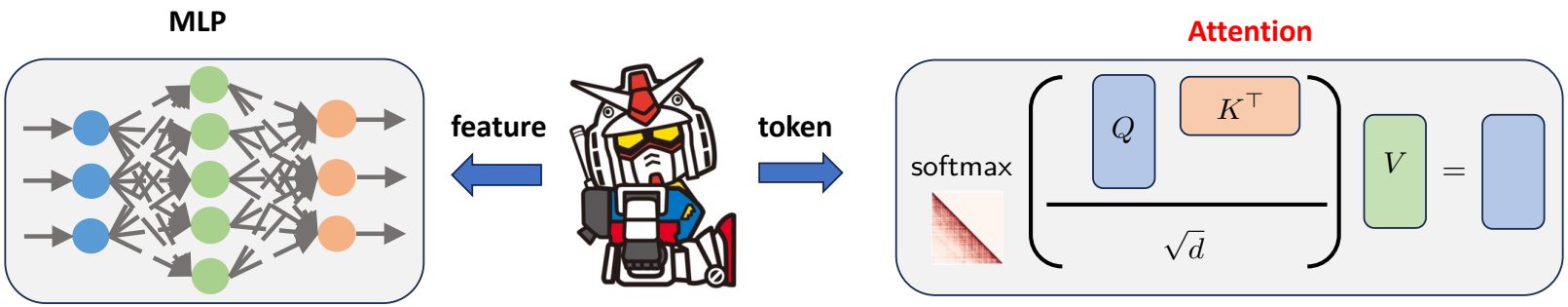
Attention searches and retrieves the values of relevant tokens

Transformers as the backbone architecture

Sequence-to-sequence:

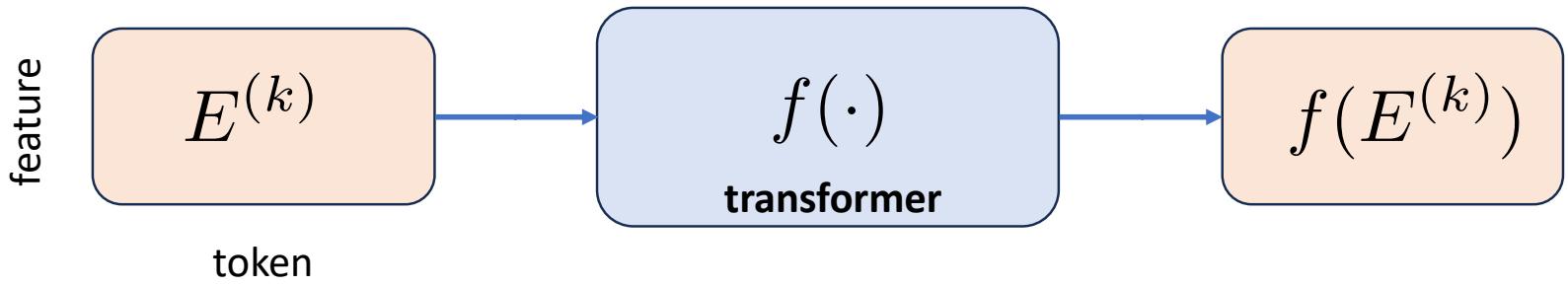


Transformer block = attention + MLP

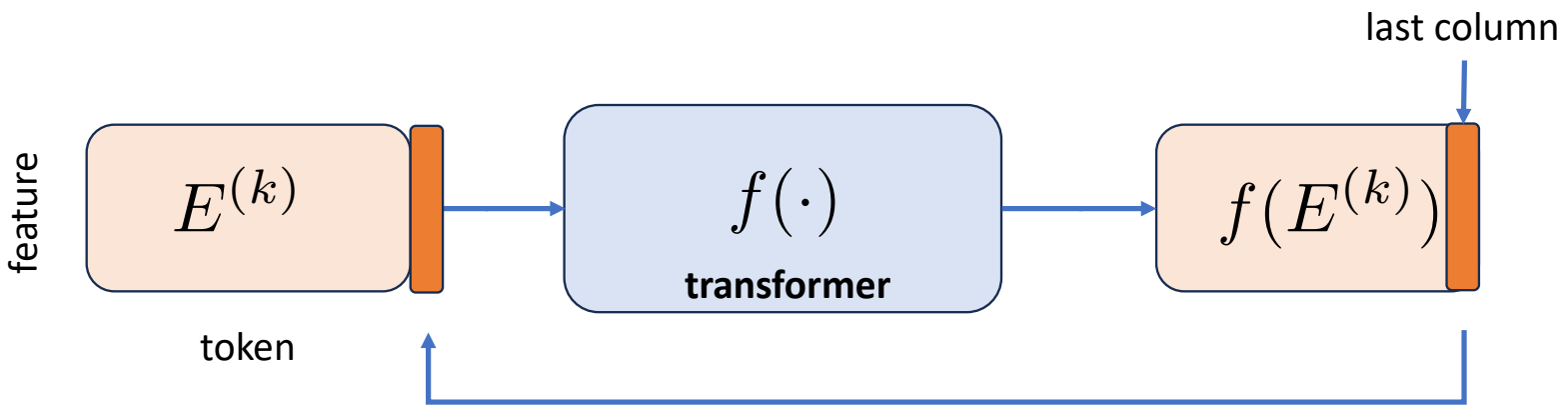


Transformers as the backbone architecture

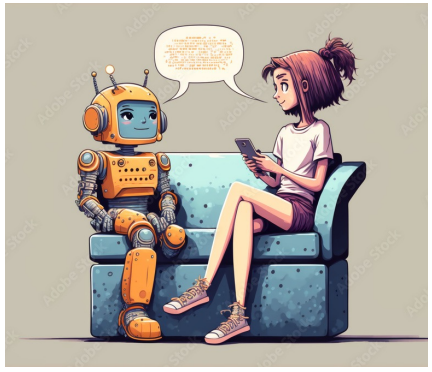
Sequence-to-sequence:

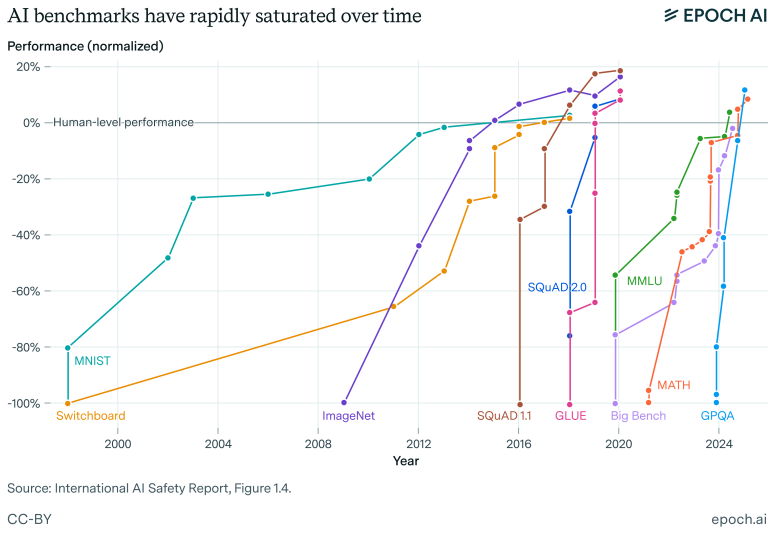


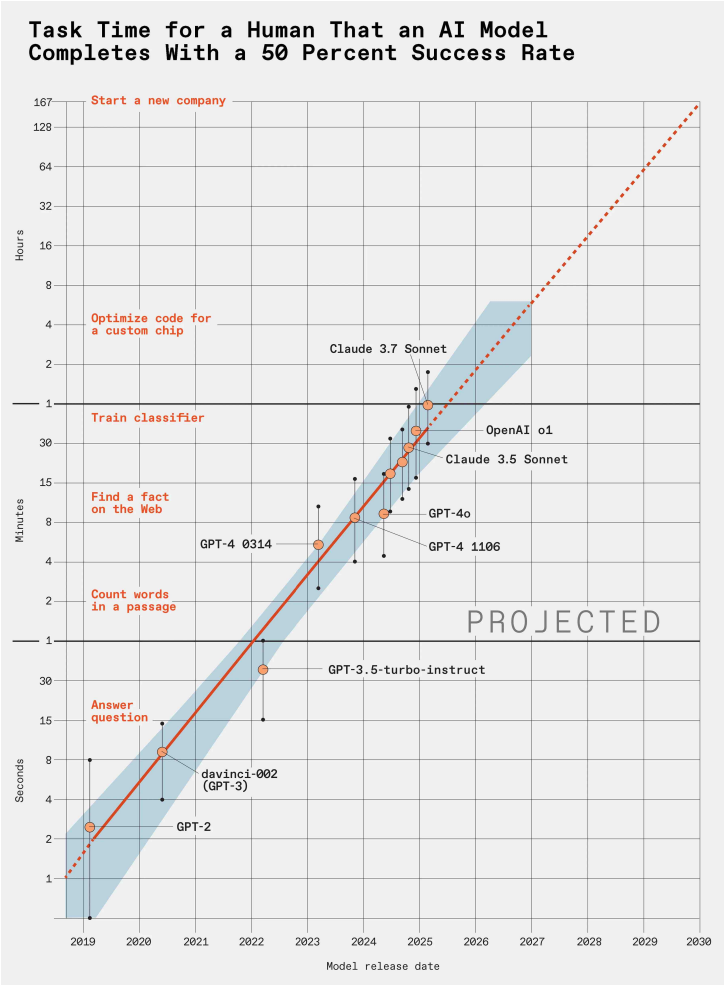
Autoregressive decoding:



The rise of large language models







LLM benchmarking shows capabilities doubling every **7 months**.

`https://spectrum.ieee.org/large-language-model-performance`

Chain-of-thought (CoT)

LLMs solve harder questions by reasoning **step-by-step**: generate intermediate reasoning steps before inferring an answer.*

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

<Question→Answer>

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

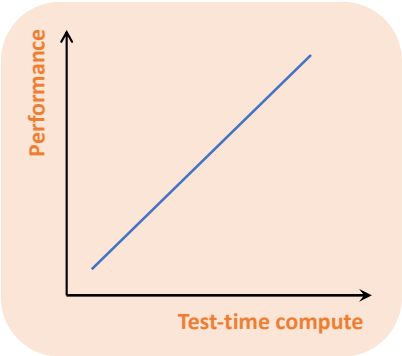
Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

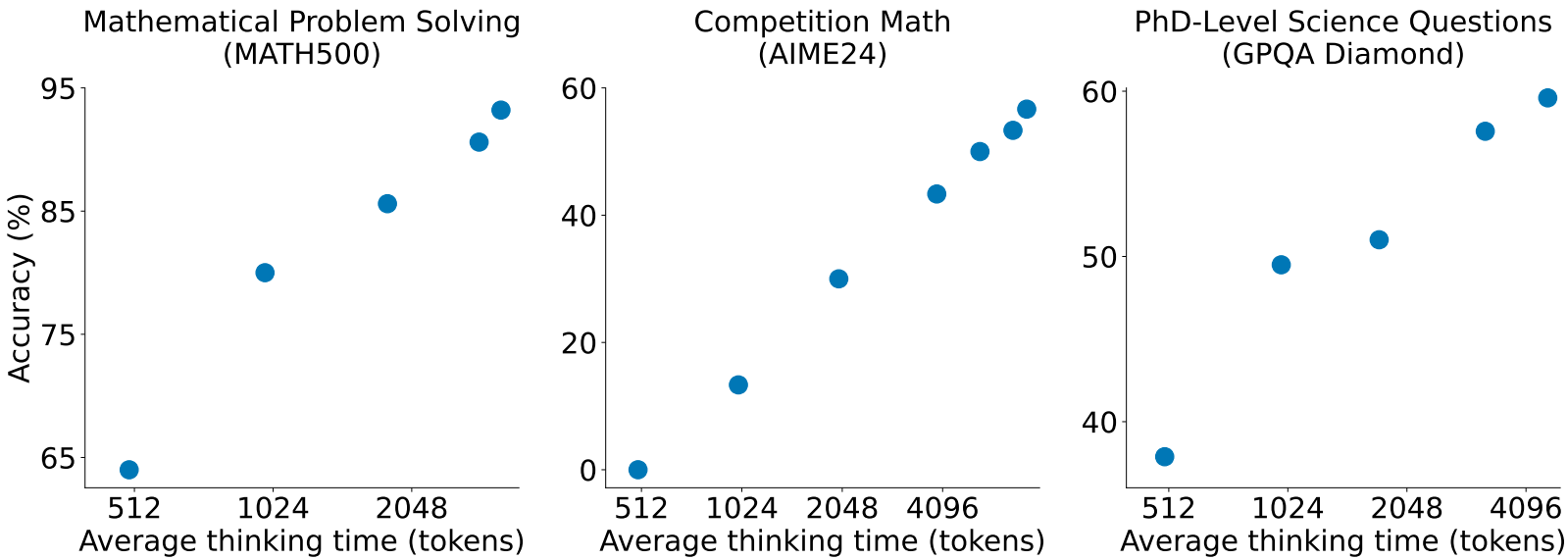
<Question→Rationale→Answer>

*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

Test-time scaling

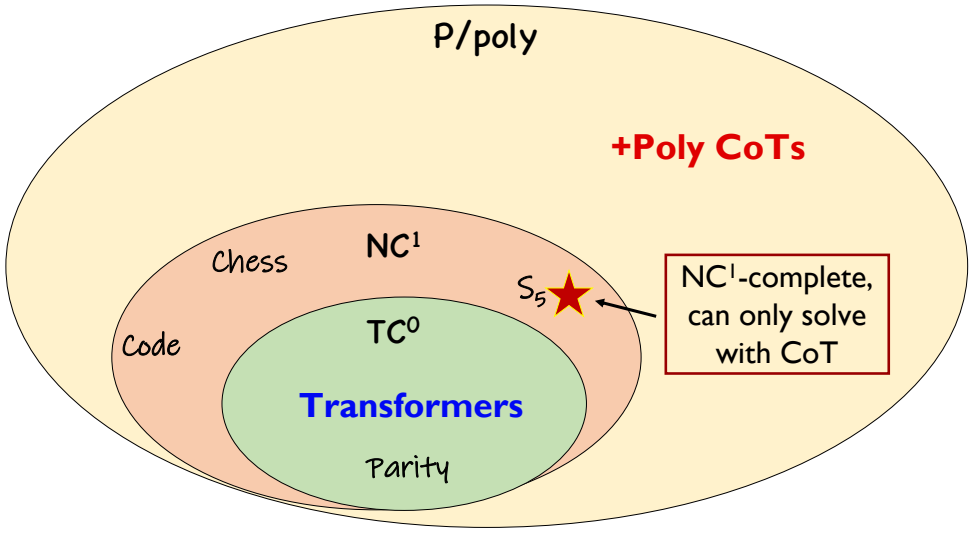


Reasoning performance improves with more test-time compute (e.g., thinking longer).



(Figure credit: s1: Simple test-time scaling)

Expressive power of CoT



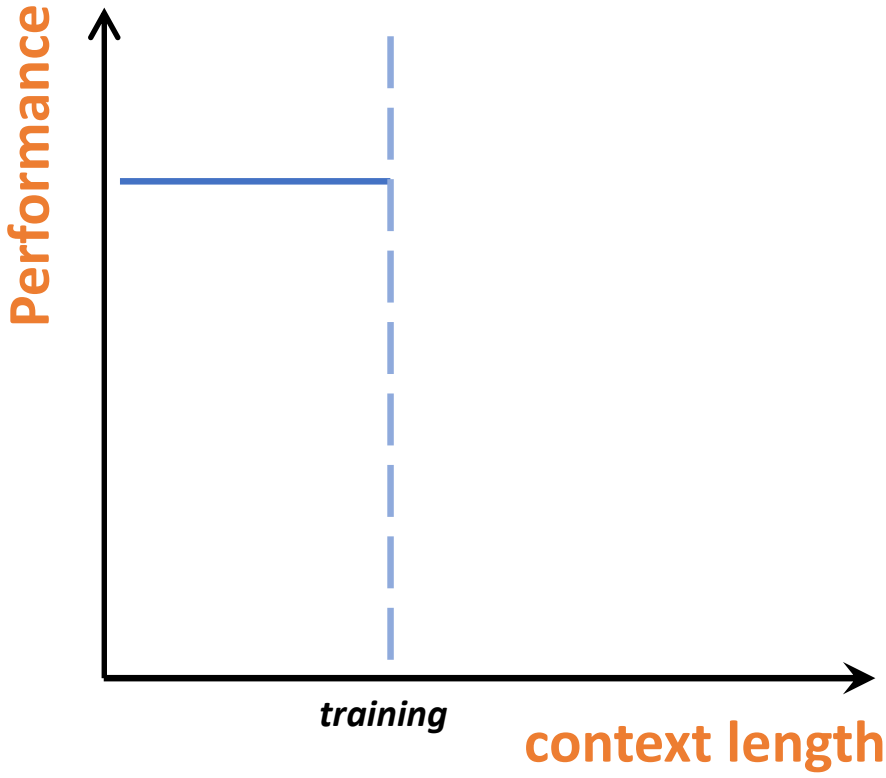
assuming $TC^0 \neq NC^1$

Expressiveness: (Li et al., 2024; Merrill et al., 2023; Feng et al., 2023...)

- without CoT $\subset \mathsf{TC}^0$;
- with linear CoTs $\supseteq \mathsf{NC}^1$;
- with polynomial CoTs $\supseteq P/\text{poly}$.

With increasing CoT lengths, transformers become more powerful!

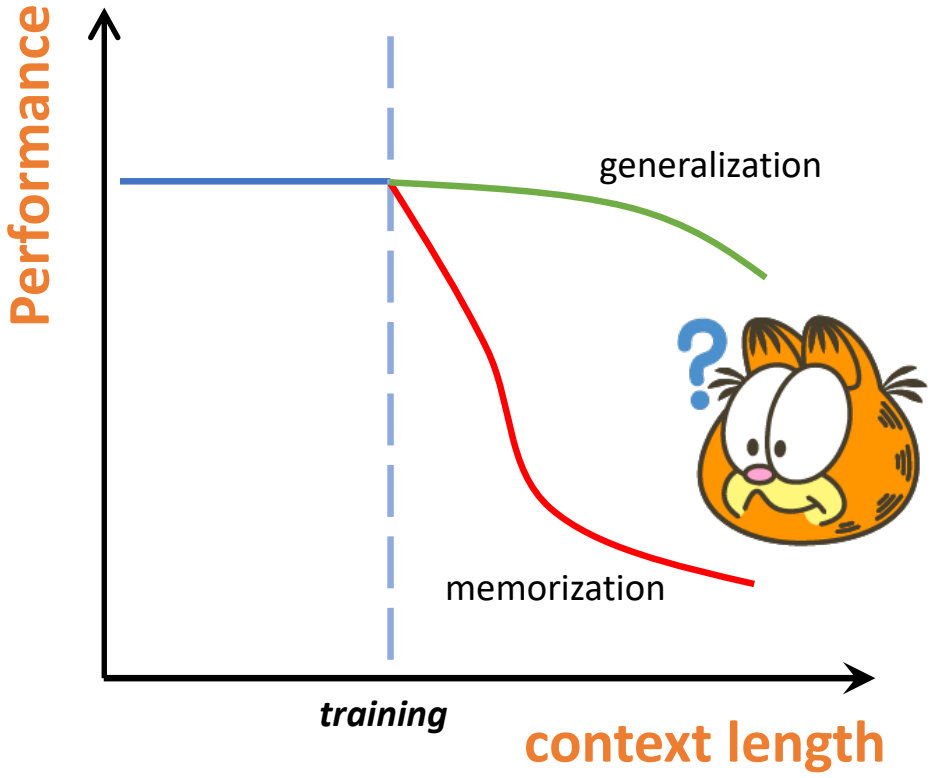
Length generalization from training perspective



Can transformers trained on shorter tasks be generalized to longer ones?

No optimization guarantees *beyond* TC^0.

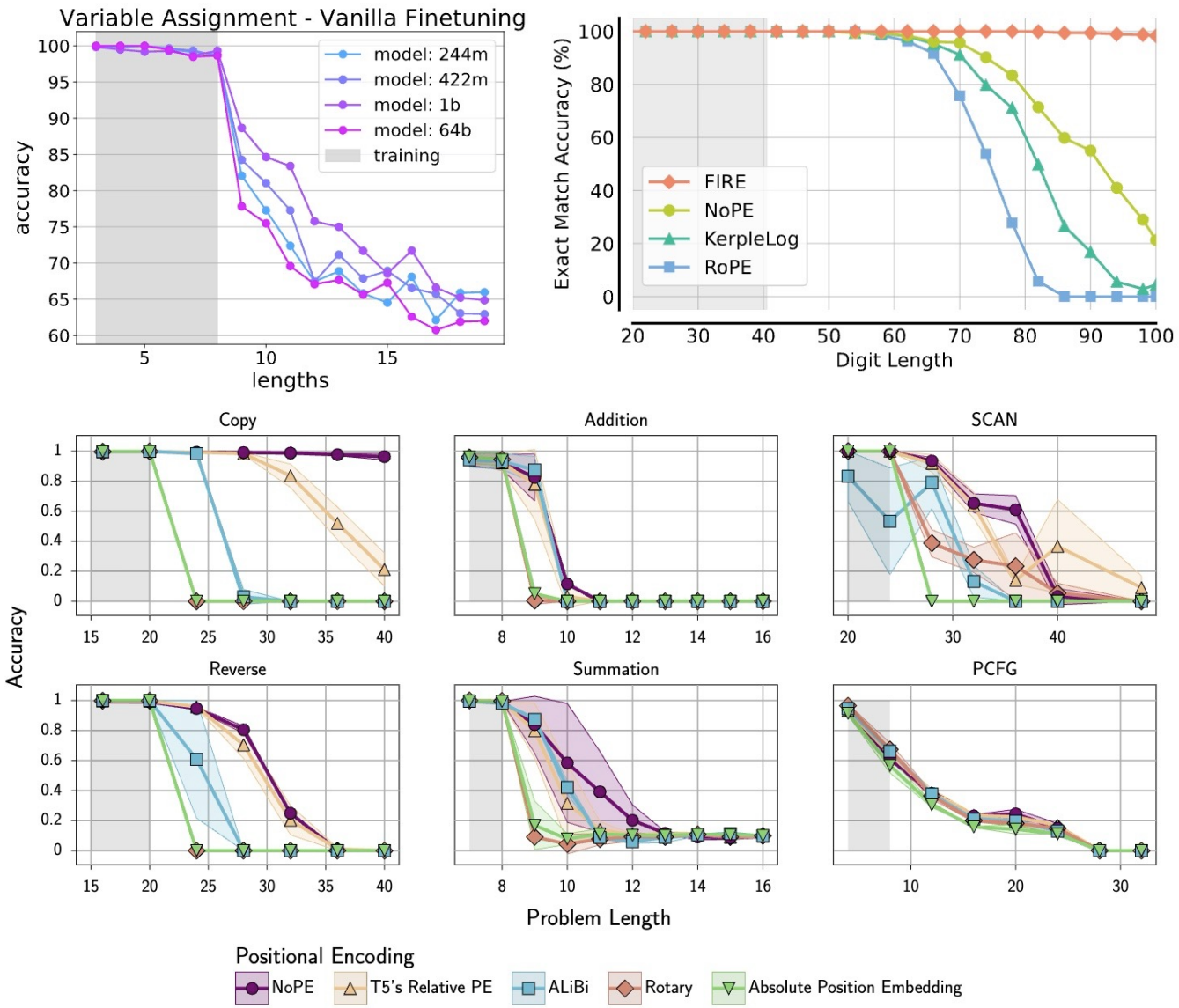
Length generalization from training perspective



Can transformers trained on shorter tasks be generalized to longer ones?

No optimization guarantees *beyond* TC^0.

Empirical evidence is mixed



(Figure credit: Anil et al., 2022; Kazemnejad et al., 2023; Zhou et al., 2024)

This talk: training dynamics of CoT reasoning

Why do *trained* transformers perform multi-step reasoning?

Optimization

Generalization

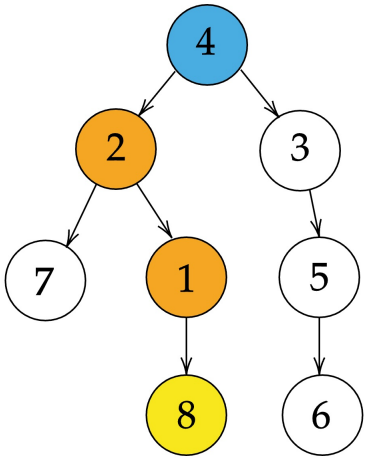
- What does gradient descent converge to?
- What is the trained attention at convergence?

- Will trained transformers generalize to unseen tasks?
- Will trained transformers generalize to longer lengths?

Take-away: one-layer transformers with CoT training can solve reasoning tasks that are otherwise out of reach

This talk: two case studies with one-layer transformers

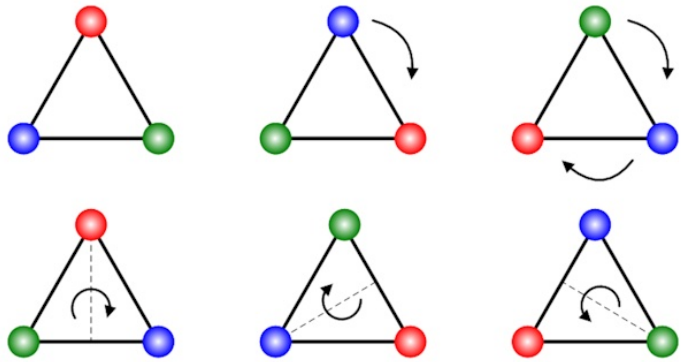
symbolic reasoning with multi-head attention



Path-finding in a tree

(Brinkman et al., 2024)

reasoning with group action with attention + MLP



LEGO

(Zhang et al., 2023)

Multi-head transformers learn symbolic multi-step reversal reasoning

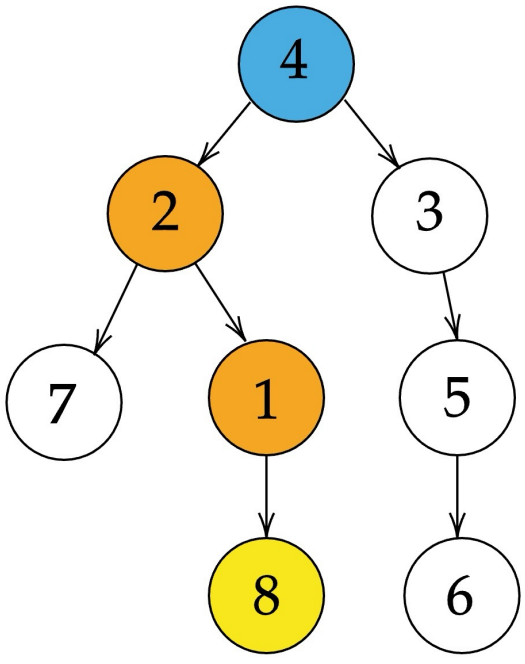
Tong Yang
CMU

Yu Huang
Penn

Yingbin Liang
OSU

"Multi-head Transformers Provably Learn Symbolic Multi-step Reasoning via Gradient Descent,"
T. Yang, Y. Huang, Y. Liang, Y. Chi, *NeurIPS 2025*

Pathfinding in trees



Prompt:

Edge set: $1 \to 8,\ 4 \to 2,\ 2 \to 7, 3 \to 5,$
$2 \to 1,\ 4 \to 3,\ 5 \to 6$

Goal: 8

Root: 4

Path-finding task:

Goal-to-root (backward):

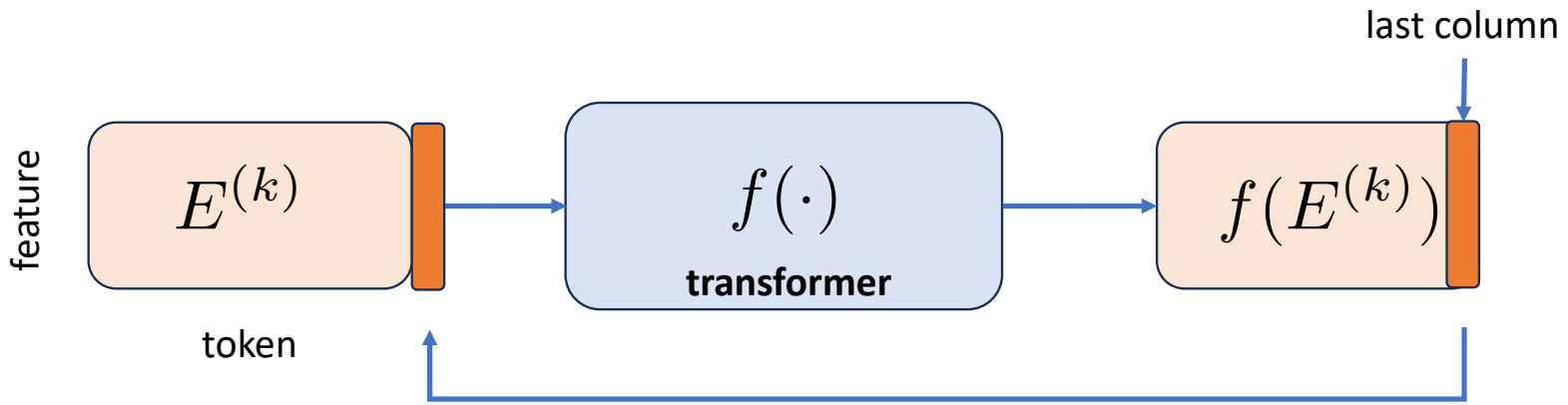
$8 \to 1 \to 2 \to 4$

Root-to-goal (forward):

$4 \to 2 \to 1 \to 8$

- **Backward** reasoning: find goal-to-root path; easy by backtracking.
- **Forward** reasoning: find the root-to-goal path; harder and needs to first solve backward task and then reverse it.

Autoregressive, multi-step, decoding



Embedding: each edge $e = (p(i), i) \in \mathcal{E}(\mathcal{T})$ is embedded by

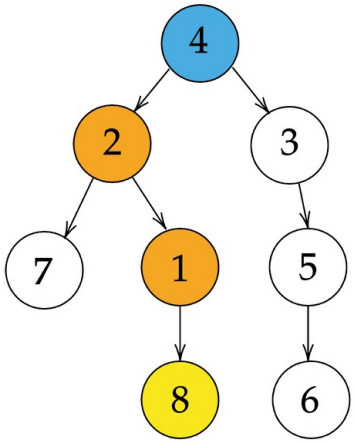
$$\begin{bmatrix} p(i) \\ i \end{bmatrix} \longrightarrow \begin{bmatrix} a_{p(i)} \\ a_i \end{bmatrix},$$

where a_i is the embedding of node i, $p(i)$ is the parent of node i.

Embedding of the input tree: (x_i, y_i)'s are the edges

$$E(\mathcal{T}) = \begin{pmatrix} X(\mathcal{T}) \\ Y(\mathcal{T}) \end{pmatrix} = \begin{pmatrix} x_1 & \dots & x_{l(\mathcal{T})} & a_0 & a_{g(\mathcal{T})} \\ y_1 & \dots & y_{l(\mathcal{T})} & a_{r(\mathcal{T})} & a_0 \end{pmatrix} \in \mathbb{R}^{2d_1 \times (l(\mathcal{T})+2)}$$

Single-head attention for one-step backward reasoning



$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[\begin{array}{ccccccc|c|c} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{array}\right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

edges r g

query

Attention searches the edge and retrieves the parent node

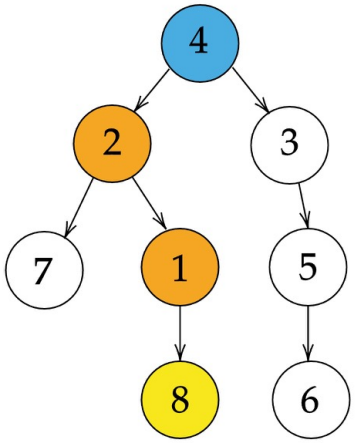
Attention output: simplified model with trainable B

$$\widehat{o}^{(k)} = \begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix} \cdot \text{softmax}\left(Y^{(k-1)\top} B x_{-1}^{(k-1)} \right),$$

where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$.

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

Single-head attention for one-step backward reasoning



$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[\begin{array}{ccccccc|c|c} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{array}\right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

edges r g

search

query

Attention searches the edge and retrieves the parent node

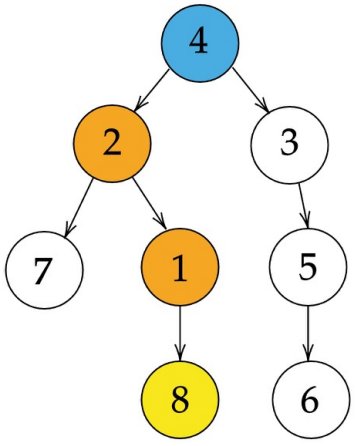
Attention output: simplified model with trainable B

$$\widehat{o}^{(k)} = \begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix} \cdot \text{softmax}\left(Y^{(k-1)\top} B x_{-1}^{(k-1)} \right),$$

where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$.

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

Single-head attention for one-step backward reasoning



$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[\begin{array}{ccccccc|c|c} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{array}\right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

Attention searches the edge and retrieves the parent node

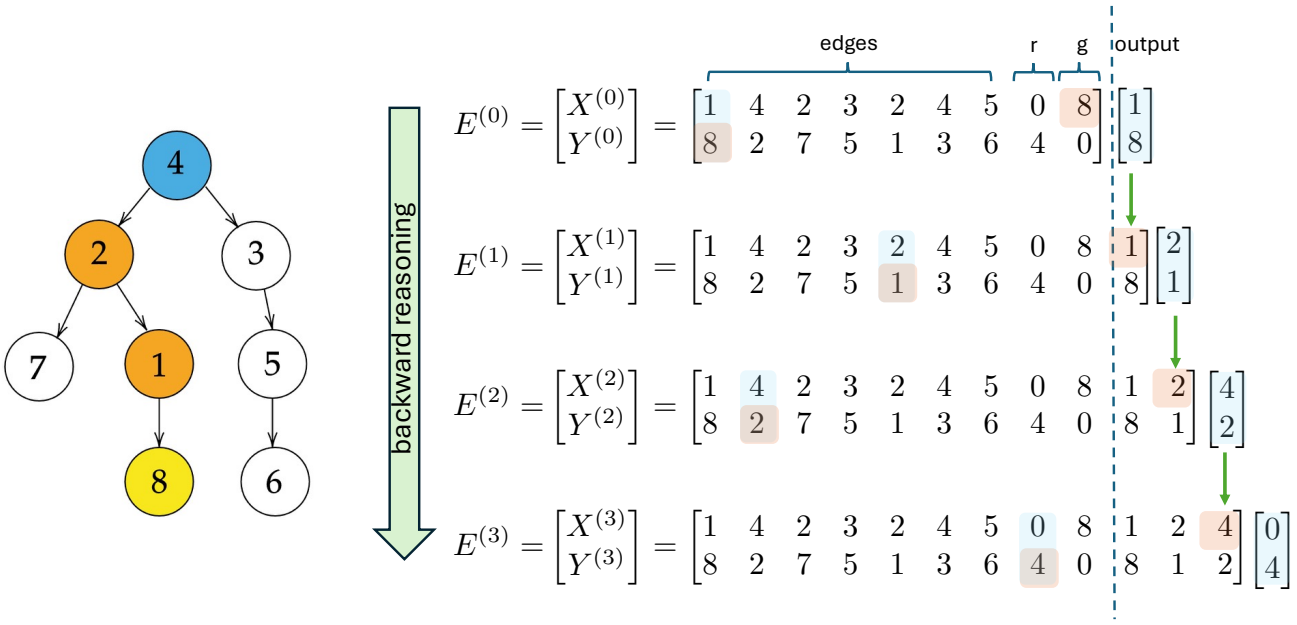
Attention output: simplified model with trainable B

$$\widehat{o}^{(k)} = \begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix} \cdot \text{softmax}\left(Y^{(k-1)\top} B x_{-1}^{(k-1)} \right),$$

where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$.

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

Single-head attention for backward reasoning

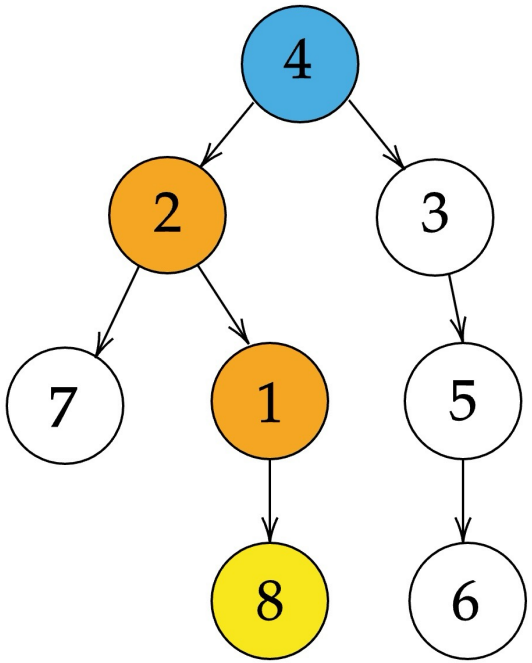


- Let $A := (a_1, \ldots, a_S) \in \mathbb{R}^{d_1 \times S}$ be the embedding matrix with linearly independent columns, and $a_0 = 0_{d_1}$.

Theorem (Construction for backward reasoning)

For any $\alpha \in \mathbb{R}$, there exists $B = B_\alpha \in \mathbb{R}^{d_1 \times d_1}$ such that $A^\top B A = \alpha I_S$. Then for any tree $\mathcal{T}$, we have $\widehat{O}(\mathcal{T}; B) \to O(\mathcal{T})$ as $\alpha \to +\infty$.

Task switching for forward reasoning



Root-to-goal (forward):

$$8 \to 1 \to 2 \to 4 \to 2 \to 1 \to 8$$

Stage 1
backtracking

Stage 2
CoT Retrieval

Task switch: To implement forward reasoning, we need to solve and automatically switch between two subtasks:

- Backtracking: find the goal-to-root path and store it in CoT;
- CoT retrieval: retrieve the root-to-goal path from CoT.

Two-head attention for forward reasoning

Embedding of the input tree with two heads:

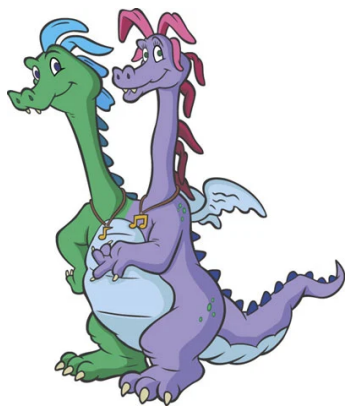
$$E_{\mathsf{r2g}}(\mathcal{T}) = \begin{pmatrix} X(\mathcal{T}) \\ Y(\mathcal{T}) \\ Z(\mathcal{T}) \end{pmatrix} = \begin{pmatrix} x_1 & \dots & x_{l(\mathcal{T})} & a_0 & a_0 \\ y_1 & \dots & y_{l(\mathcal{T})} & a_{r(\mathcal{T})} & a_{g(\mathcal{T})} \\ s_f & \dots & s_f & s_f & s_b \end{pmatrix},$$

where $s_f, s_b \in \mathbb{R}^{d_2}$ are stage token embeddings.

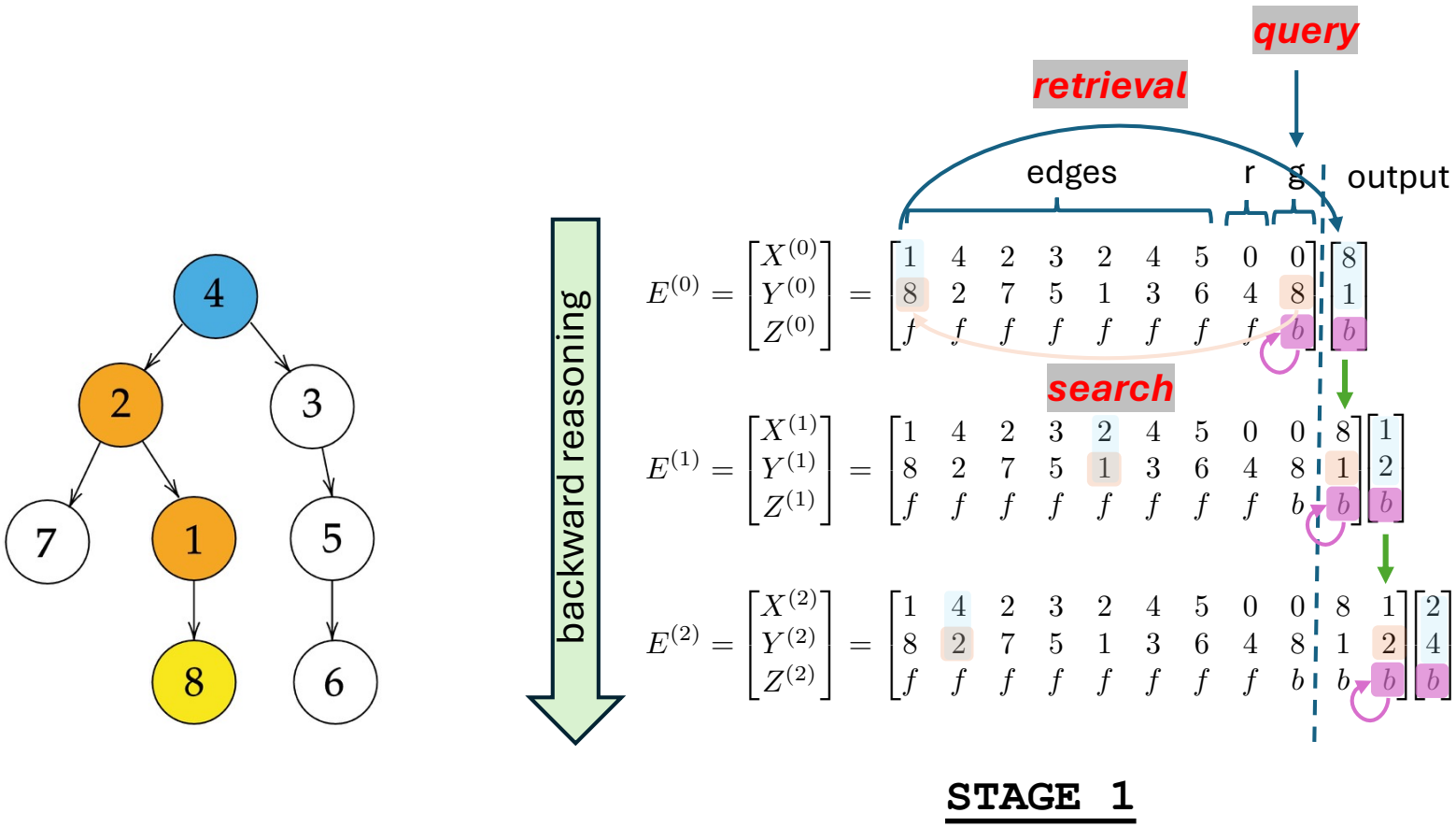
Attention output: simplified model with trainable $\{B_i, C_i\}_{i\in[3]}$:

$$\widehat{o}^{(k)} = \begin{pmatrix} \begin{pmatrix} Y^{(k-1)} \\ X^{(k-1)} \end{pmatrix} \cdot \mathsf{softmax}\left(X^{(k-1)\top} B_1 y_{-1}^{(k-1)} + Y^{(k-1)\top} B_2 y_{-1}^{(k-1)} + Z^{(k-1)\top} B_3 z_{-1}^{(k-1)} \right) \\ Z^{(k-1)} \cdot \mathsf{softmax}\left(X^{(k-1)\top} C_1 y_{-1}^{(k-1)} + Y^{(k-1)\top} C_2 y_{-1}^{(k-1)} + Z^{(k-1)\top} C_3 z_{-1}^{(k-1)} \right) \end{pmatrix}.$$

- The first head performs edge retrieval;
- The second head controls the task switch.

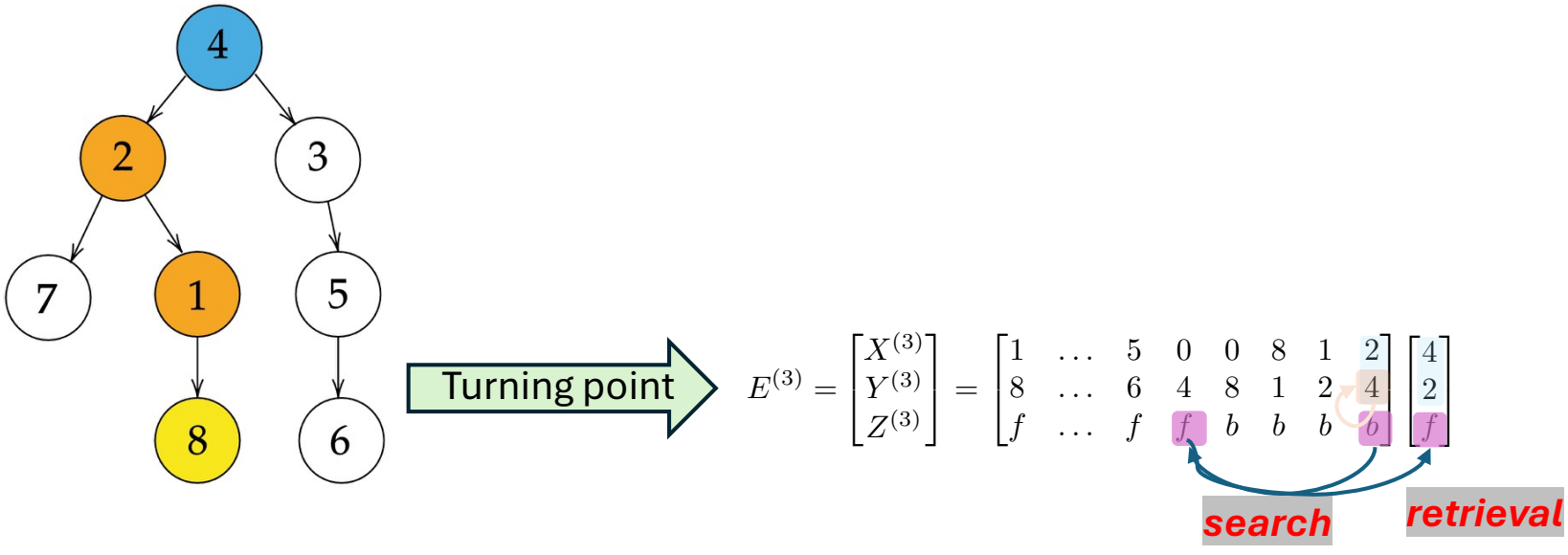


Two-head attention for forward reasoning: Stage 1



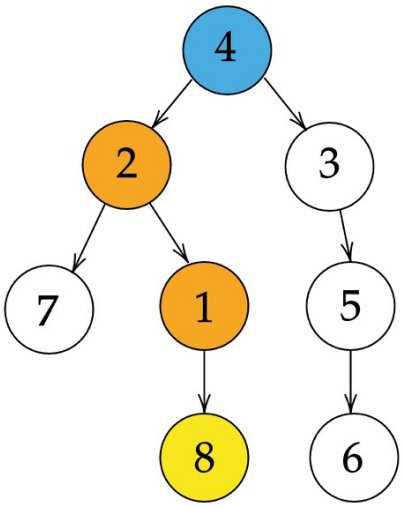
- First head: attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(n^{(k-1)}, p(n^{(k-1)}))$.
- Second head: attend to and output (its own) stage signal s_b.

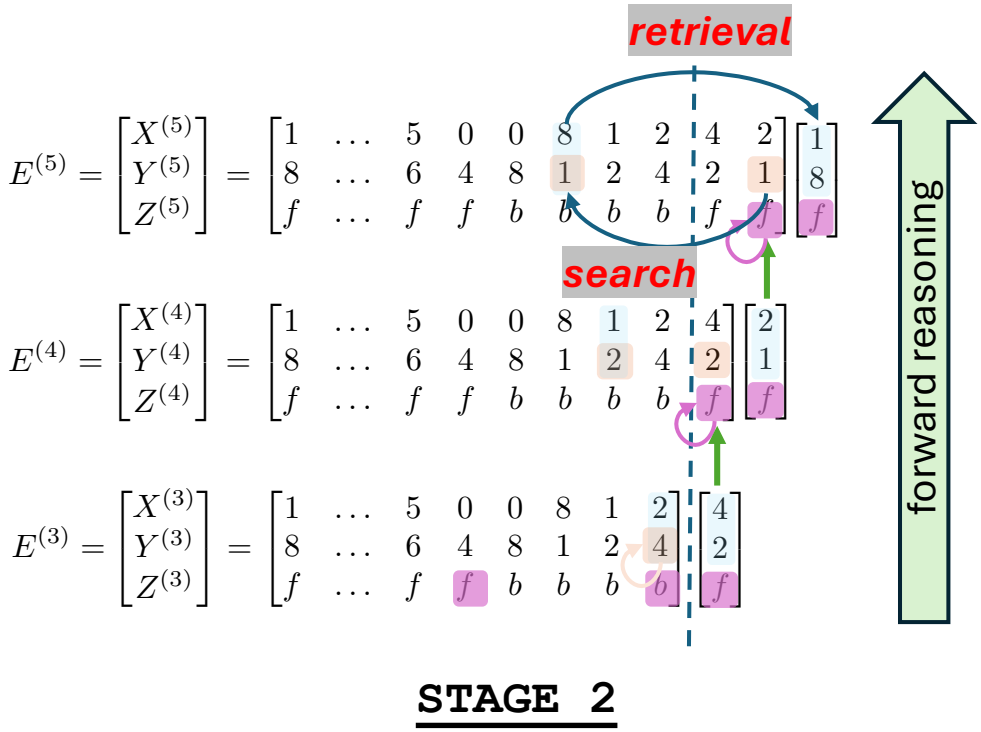
Two-head attention for forward reasoning: Turning Point



- First head: attend to $(y_{-1}^{(k-1)}, x_{-1}^{(k-1)})$ (itself) via null node embedding at root, and output $(x_{-1}^{(k-1)}, y_{-1}^{(k-1)})$.
- Second head: attend to and output stage signal s_f via null node embedding at root.

Two-head attention for forward reasoning: Stage 2





STAGE 2

- First head: attend to $(n^{(k-1)}, p(n^{(k-1)}))$ generated in *STAGE 1*, and output $(p(n^{(k-1)}), n^{(k-1)})$.
- Second head: attend to and output (its own) stage signal s_f.

Construction for forward reasoning

- Denote $\widetilde{A} = (a_0, a_1, \ldots, a_S) \in \mathbb{R}^{d_1 \times (S+1)}$ and $\widetilde{S} = (s_f, s_b)$ be the embedding matrix with linearly independent columns.

Theorem (Construction for forward reasoning)

There exist $\theta = \{B_i, C_i\}_{i \in [3]}$ *that satisfies*

$$\widetilde{A}^\top B_1 A = -a\alpha_1 \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}, \; A^\top B_2 A = \alpha_1 I_S, \; \widetilde{S}^\top B_3 \widetilde{S} = \alpha_1 \begin{pmatrix} -b_1 & b_2 \\ b_1 & -b_2 \end{pmatrix},$$

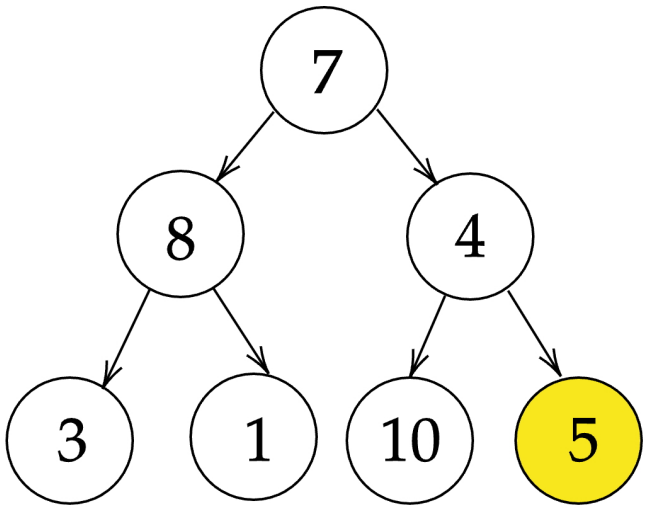
$$\widetilde{A}^\top C_1 A = \alpha_2 \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}, \; A^\top C_2 A = \alpha_2 I_S, \; \widetilde{S}^\top C_3 \widetilde{S} = \alpha_2 \begin{pmatrix} c_1 & -c_2 \\ -c_1 & c_2 \end{pmatrix}.$$

When $a \in (0, 1], b_1 > 0, b_2 \in (0, a/2)$ *and* $c_1 > 0$, $c_2 \in (0, 1/2)$, *we have* $\widehat{O}_{r2g}(\mathcal{T}; \theta) \to O_{r2g}(\mathcal{T})$ *as* $\alpha_1, \alpha_2 \to +\infty$.

- Ensures the feature interplays to enable autonomous switching.

Assumptions for training dynamics

Training distribution: Let $m \geq 3$ and $S \geq 2^{m+1} - 1 =: N$. The training distribution P_{train} is the **uniform distribution** over **perfect binary trees** $\mathcal{T} = (\mathcal{V}(\mathcal{T}), \mathcal{E}(\mathcal{T}), g(\mathcal{T}))$ of depth m with distinct nodes chosen from $[S]$ and a goal node $g(\mathcal{T})$ being one of the leaf nodes.



Orthonormal embeddings: (i) $\{a_i\}_{i \in [S]}$ are orthonormal vectors in $\mathbb{R}^{d_1}$; (ii) s_f, s_b are orthonormal vectors in $\mathbb{R}^{d_2}$.

Training objective

Autoregressive supervised learning: at each step $k \geq 2$, the input

$$E_{\text{train}}^{(k-1)}(\mathcal{T}) = (E_{\text{train}}^{(k-2)}(\mathcal{T}), o^{(k-1)}(\mathcal{T}))$$

is the concatenation of the input at step $k-1$ and the $(k-1)$-th column of the ground truth output. The output is

$$\widehat{o}_{\text{train}}^{(k)}(\mathcal{T};\theta) = f(E_{\text{train}}^{(k-1)}(\mathcal{T});\theta)_{:,-1}.$$

CoT training loss:

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{2}\,\mathbb{E}_{\mathcal{T}\sim P_{\text{train}}} \|O(\mathcal{T}) - \widehat{O}_{\text{train}}(\mathcal{T};\theta)\|_{\text{F}}^2.$$

Gradient descent: we initialize all parameters to be zero, and

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}_{\text{train}}(\theta^{(t)}), \quad \forall t \geq 0.$$

Theory of optimization

Theorem (Training dynamics)

We use zero initialization. For learning rate η and ϵ that are sufficiently small, we reach

$$\mathcal{L}_{\text{train}}(\theta^{(t)}) \leq \epsilon$$

as soon as the number of iterations satisfies

- *For backward reasoning*

$$T = \mathcal{O}\left(\frac{mS}{\eta\epsilon}\log\left(\frac{Nm}{\epsilon}\right)\right).$$

- *For forward reasoning*

$$T = \mathcal{O}\left(\frac{S}{\eta}\left(\frac{m}{\epsilon}\right)^{3/2}\log\left(\frac{N}{\epsilon}\right)\right).$$

- Backward reasoning converges at a rate of $\widetilde{\mathcal{O}}(1/\epsilon)$, and forward reasoning converges at a rate of $\widetilde{\mathcal{O}}(1/\epsilon^{3/2})$.

Theory of generalization

Generalization error: on unseen tree $\mathcal{T}$

$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta) = \frac{1}{2}\|O(\mathcal{T}) - \widehat{O}(\mathcal{T};\theta)\|_{\text{F}}^2.$$

Theorem (Generalization)

Given any tree $\mathcal{T}$ at test time with $\widetilde{N}$ distinct nodes chosen from $[S]$ and a path length $\widetilde{m}$, there exists small enough $\epsilon_0 = \epsilon_0(m, \widetilde{N}, \widetilde{m}) > 0$ such that for any $\epsilon \in (0, \epsilon_0]$, when the training less $\mathcal{L}_{\text{train}}(\theta^{(t)}) \le \epsilon$,

- *For backward reasoning,*

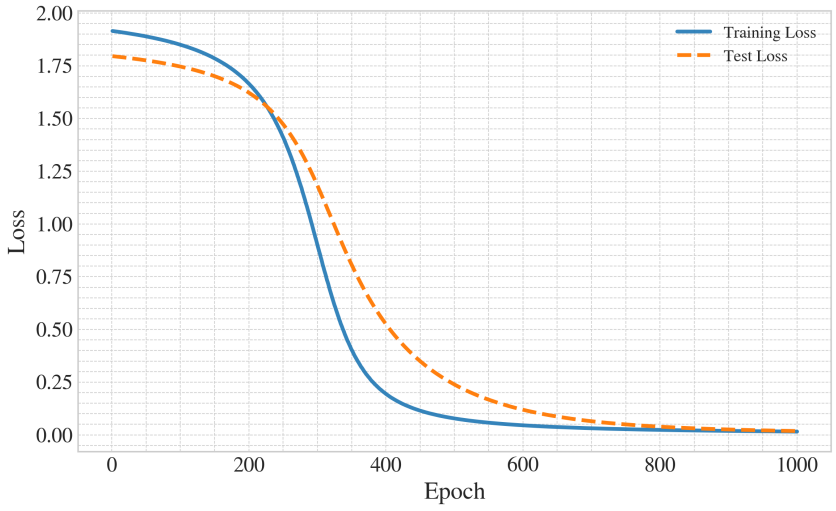
$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta^{(t)}) \le 4\max\left\{\left(\frac{\widetilde{N}+\widetilde{m}-1}{N+m-2}\right)^2, 1\right\}\cdot\frac{\widetilde{m}}{m}\epsilon.$$

- *For forward reasoning,*

$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta) \le 4\max\left\{\left(\frac{\widetilde{N}+2\widetilde{m}-1}{N+2m-1}\right)^2, \left(\frac{\widetilde{N}}{N}\right)^2, 1\right\}\epsilon.$$

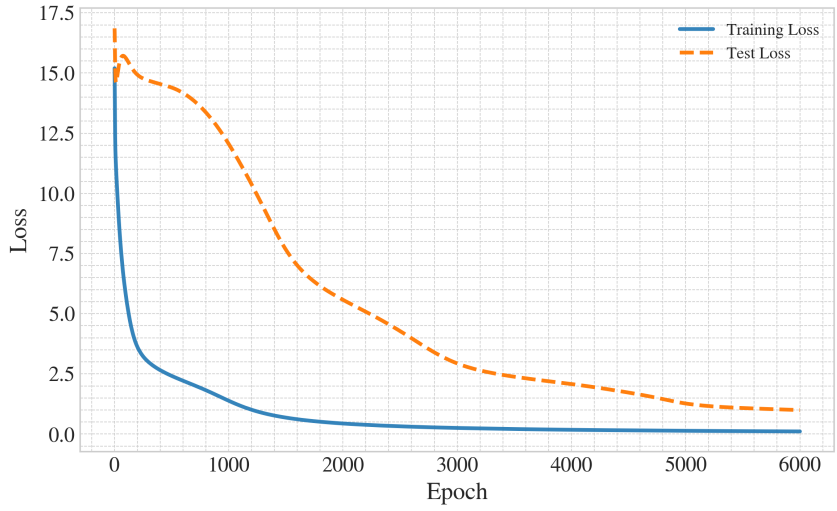
- Better length generalization on unseen trees for small training error.

Numerical experiments



Backward reasoning

$m = 4,\ S = 31$



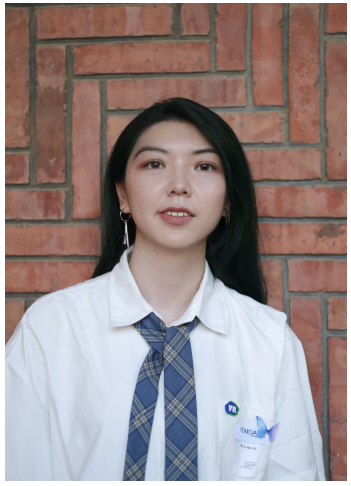
Forward reasoning

$m = 3,\ S = 25$

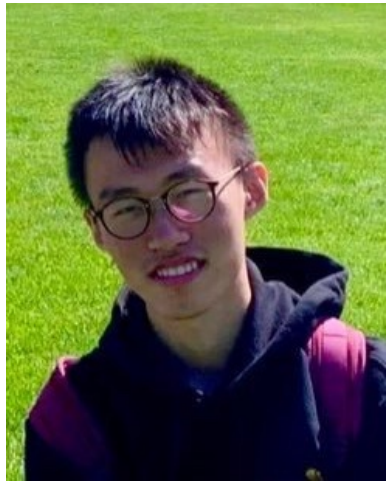
- The training dynamics of attention matrices also follow our analysis.

Transformer generalizes, rather than memorizes.

Transformers learn multi-step compositional reasoning with length generalization



Yu Huang*
Penn



Zixin Wen*
CMU

Aarti Singh
CMU

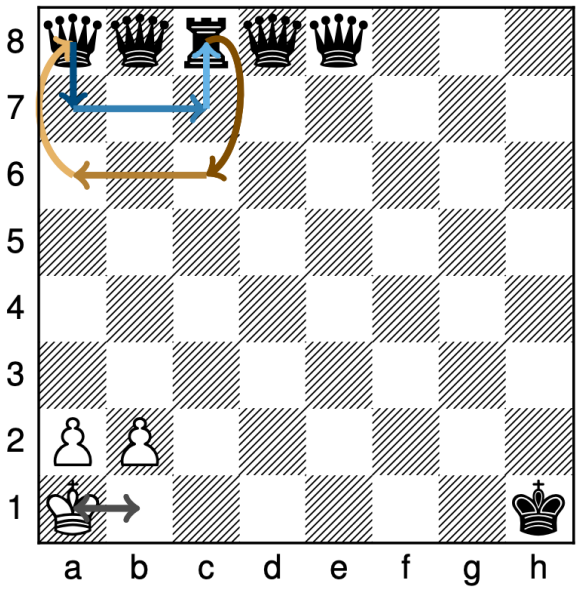
Yuxin Chen
Penn

"Transformers Provably Learn Chain-of-Thought Reasoning with Length Generalization,"
Y. Huang*, Z. Wen*, A. Singh, Y. Chi, Y. Chen, *NeurIPS 2025*

State tracking

Given an initial state and a sequence of *actions*, compute the final state.

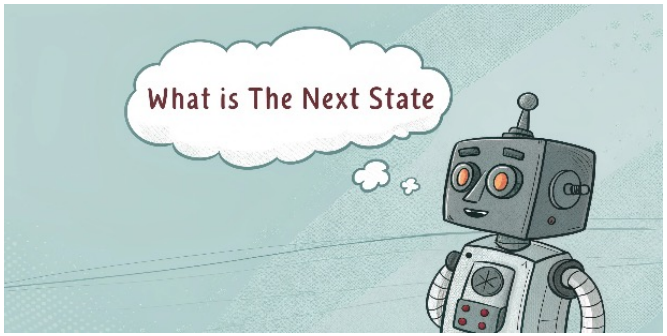
Playing Chess



Images modified from Merrill et al., 2024

Code evaluation

```
x = [0, 0, 1, 0, 0]
x[1], x[3] = x[3], x[1] # Swap 1, 3
```



Entities tracking

Alice, Bob, Carl, Dan, and Emma each have a coin. All are dimes except Carl's. Alice and Carl trade coins.

State tracking task: LEGO

LEGO sentence (Zhang et al., 2022): with answer up to $m < L$

$$\underbrace{\boxed{x_1 = g_1(x_0)}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})}}_{\text{predicate clauses}} \qquad \underbrace{\boxed{x_0 = y_0}, \ \ldots, \ \boxed{x_m = y_m}}_{\text{answer clauses}}$$

with action $g_i \in \mathcal{G}$, value $y_i \in \mathcal{Y}$, variable $x_i \in \mathcal{X}$.

Goal: correctly calculate the last value $y_L = g_L(g_{L-1}(\cdots g_1(y_0)))$.

$$x_0 \xrightarrow{g_1} x_1 \xrightarrow{g_2} x_2 \xrightarrow{g_3} \cdots \xrightarrow{g_L} x_L, \quad \text{starting with } x_0 = y_0.$$

State tracking task: LEGO

LEGO sentence (Zhang et al., 2022): with answer up to $m < L$

$$\underbrace{\boxed{x_1 = g_1(x_0)}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})}}_{\text{predicate clauses}} \qquad \underbrace{\boxed{x_0 = y_0}, \ \ldots, \ \boxed{x_m = y_m}}_{\text{answer clauses}}$$

with action $g_i \in \mathcal{G}$, value $y_i \in \mathcal{Y}$, variable $x_i \in \mathcal{X}$.

Goal: correctly calculate the last value $y_L = g_L(g_{L-1}(\cdots g_1(y_0)))$.

$$x_0 \xrightarrow{g_1} x_1 \xrightarrow{g_2} x_2 \xrightarrow{g_3} \cdots \xrightarrow{g_L} x_L, \quad \text{starting with } x_0 = y_0.$$

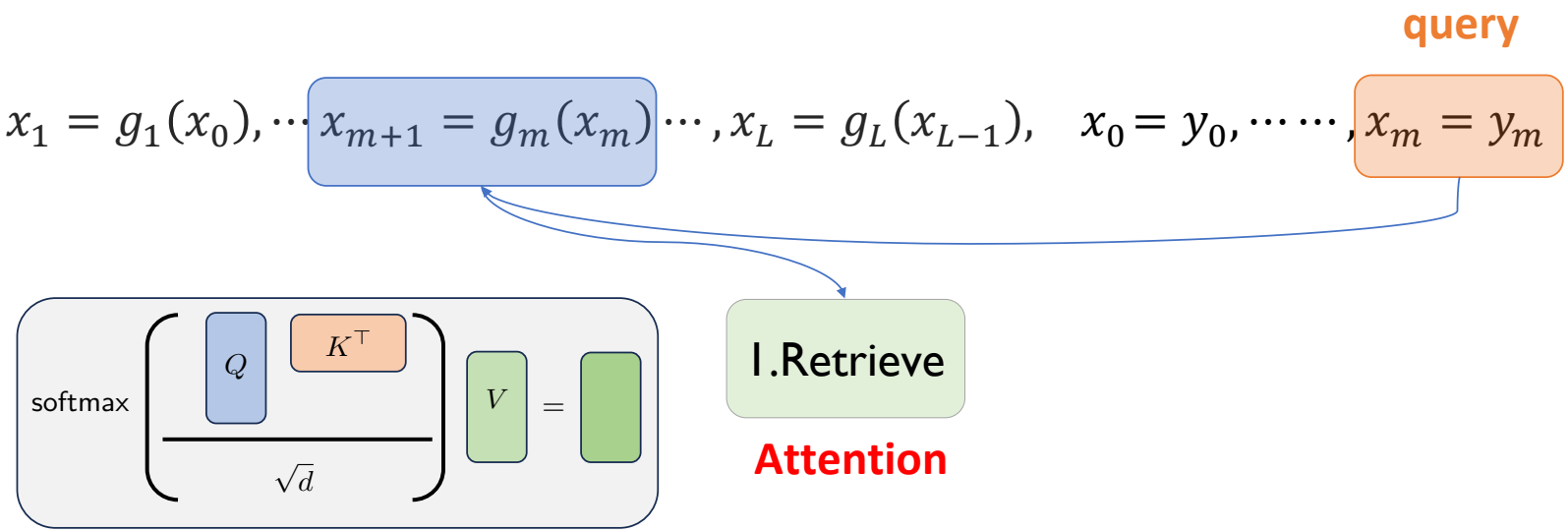
Example: modulo sum

$$x_1 = x_0 \oplus_5 4, \ x_2 = x_1 \oplus_5 2, \quad x_0 = 0, x_1 = ?, x_2 = ?$$

where

$$x_2 = x_1 \underbrace{\oplus_5 2}_{g_2} = (x_0 \underbrace{\oplus_5 4}_{g_1}) \oplus_5 2 = (\underbrace{0}_{y_0} \oplus_5 4) \oplus_5 2 \equiv 1 \pmod 5.$$

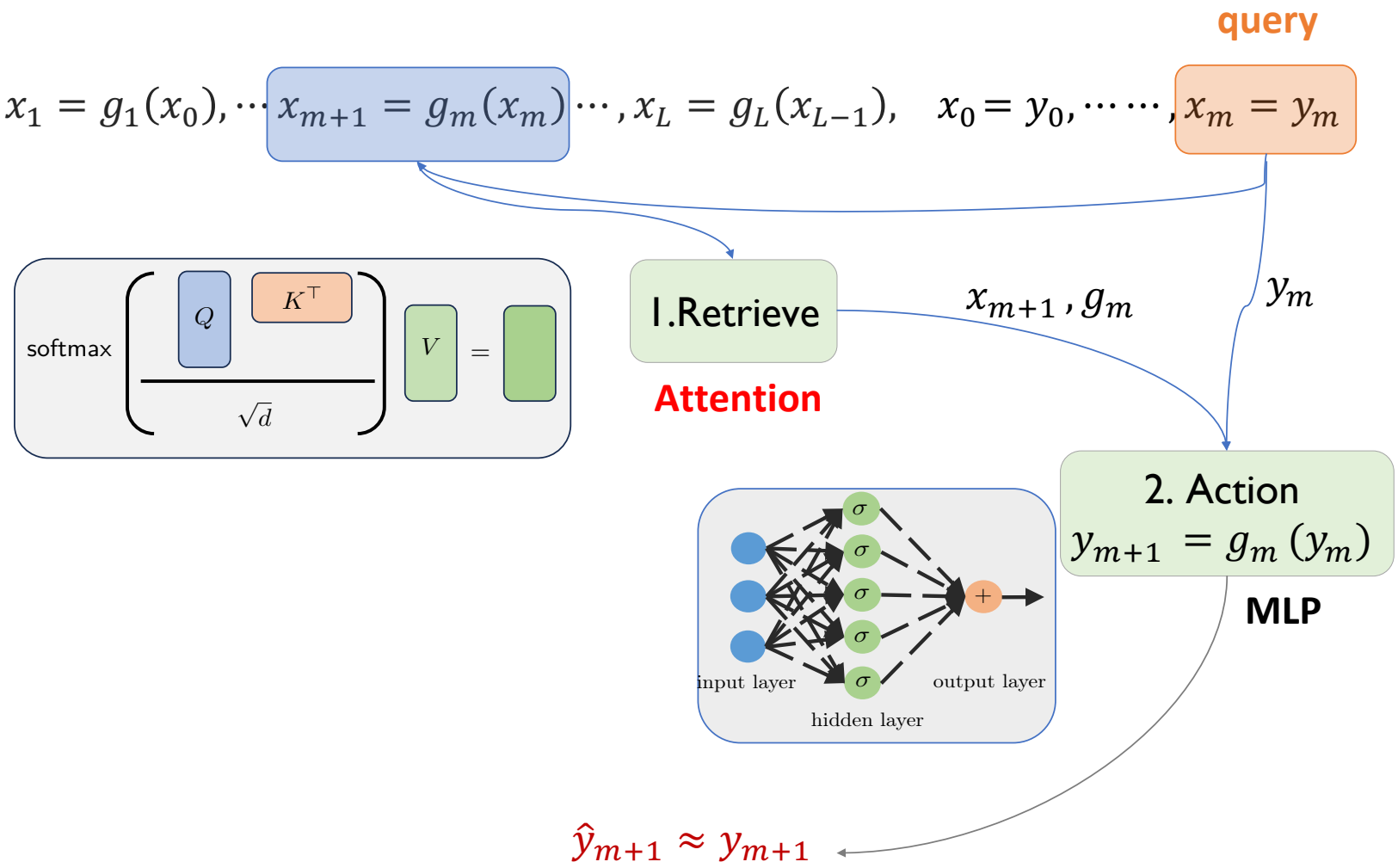
One-layer attention + MLP model



$$\hat{y}_{m+1} \approx y_{m+1}$$

Model mechanism: Attention (retrieval)

One-layer attention + MLP model



Model mechanism: FFN (action) ∘ Attention (retrieval)

Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

Training objective: minimize the cross-entropy loss for *next-answer* prediction via gradient descent

$$\mathcal{L}(F) = \mathbb{E}\Big[\sum_{m=1}^{L} -\log p_F(y_{m+1} \mid \mathbf{Z}^{L,m})\Big]$$

where $d = |\mathcal{X}| + |\mathcal{G}| + |\mathcal{Y}|$ (roughly embedding dimension).

Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

Training objective: minimize the cross-entropy loss for *next-answer* prediction via gradient descent

$$\mathcal{L}(F) = \mathbb{E}\Big[\sum_{m=1}^{L} -\log p_F(y_{m+1} \mid \mathbf{Z}^{L,m})\Big]$$

where $d = |\mathcal{X}| + |\mathcal{G}| + |\mathcal{Y}|$ (roughly embedding dimension).

Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

Training objective: minimize the cross-entropy loss for *next-answer* prediction via gradient descent

$$\mathcal{L}(F) = \mathbb{E}\Big[\sum_{m=1}^{L} -\log p_F(y_{m+1} \mid \mathbf{Z}^{L,m})\Big]$$

where $d = |\mathcal{X}| + |\mathcal{G}| + |\mathcal{Y}|$ (roughly embedding dimension).

Uniform attention initialization: $\mathbf{Attn}_{\text{ans},m\to\mathbf{k}}$ is the same for all clauses

- $\mathbf{Attn}_{\text{ans},m\to\mathbf{k}}$: attention score from $\mathbf{Z}_{\text{ans},m} \to \mathbf{Z}_{\mathbf{k}}$;

Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

Training objective: minimize the cross-entropy loss for *next-answer* prediction via gradient descent

$$\mathcal{L}(F) = \mathbb{E}\Big[\sum_{m=1}^{L} -\log p_F(y_{m+1} \mid \mathbf{Z}^{L,m})\Big]$$

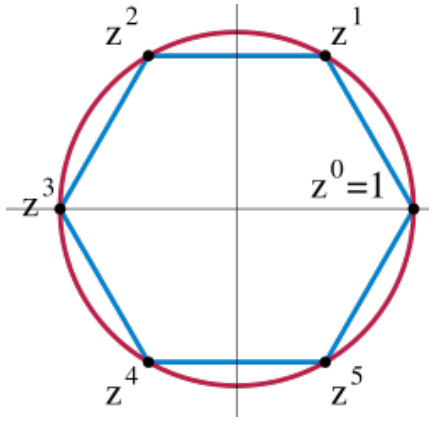
where $d = |\mathcal{X}| + |\mathcal{G}| + |\mathcal{Y}|$ (roughly embedding dimension).

Uniform attention initialization: $\mathbf{Attn}_{\text{ans},m\to\mathbf{k}}$ is the same for all clauses

- $\mathbf{Attn}_{\text{ans},m\to\mathbf{k}}$: attention score from $\mathbf{Z}_{\text{ans},m} \to \mathbf{Z}_{\mathbf{k}}$;

Tracking how attention concentrates on the key clauses and **quantify** the concentration *at loss convergence*

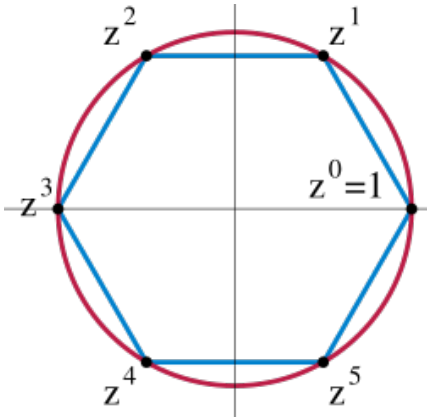
Two group actions

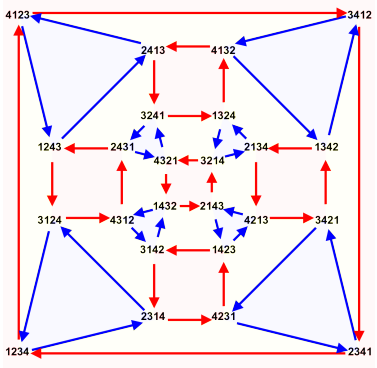


Simple transitive action: transitive and $\forall y, y'$, there is a unique $g \in \mathcal{G}$ s.t., $g(y) = y'$

- Cyclic group (mod n)
- The task is in TC^0

Two group actions





Simple transitive action: transitive and $\forall y, y'$, there is a unique $g \in \mathcal{G}$ s.t., $g(y) = y'$

- Cyclic group (mod n)
- The task is in TC^0

Symmetry action S_n: all permutations (bijections) of n elements, with composition as the operation

- $\mathcal{G}_{y \to y'} = \{g \in \mathcal{G} : \ g(y) = y'\}$ is large
- S_n with $n \geq 5$ is in NC^1 (harder)

Length generalization for simple transitive group

Theorem (Short-to-poly length generalization)

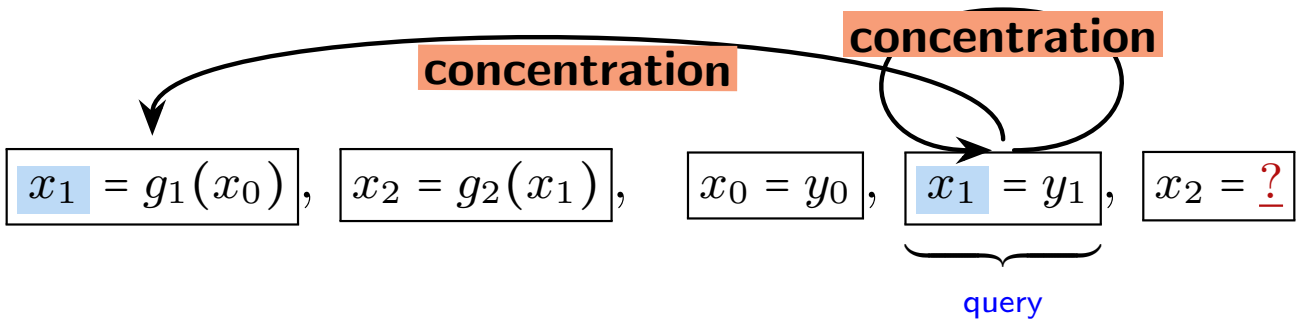
If $\mathcal{G}$ is simply transitive, then the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq \mathrm{poly}(d)$.

Length generalization for simple transitive group

Theorem (Short-to-poly length generalization)

If $\mathcal{G}$ is simply transitive, then the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq \mathrm{poly}(d)$.

Mechanism: FFN perfectly captures action; while attention learns to concentrate on the **relevant** clauses.



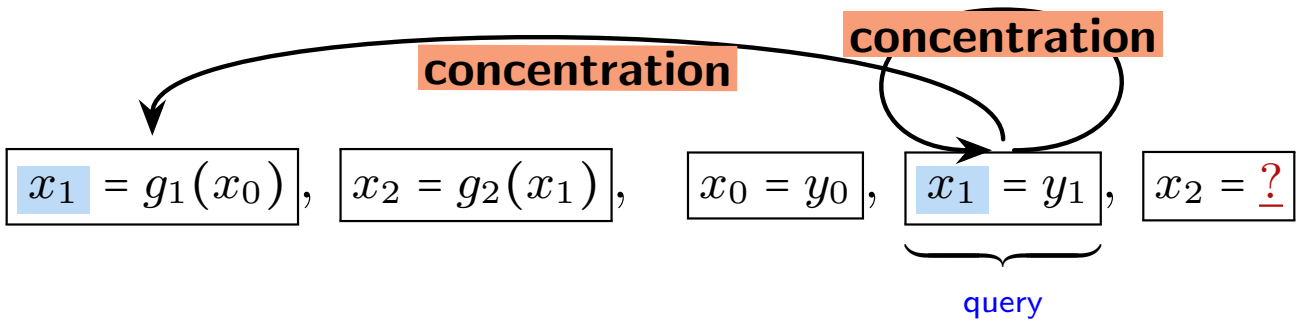
Attention concentration: $\mathbf{Attn}_{\text{ans},\ell\to\text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell\to\text{ans},\ell} = 1 - \frac{1}{\mathrm{poly}\, d}$

Length generalization for simple transitive group

Theorem (Short-to-poly length generalization)

If $\mathcal{G}$ is simply transitive, then the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq \mathrm{poly}(d)$.

Mechanism: FFN perfectly captures action; while attention learns to concentrate on the **relevant** clauses.



align the shared variable x_ℓ.

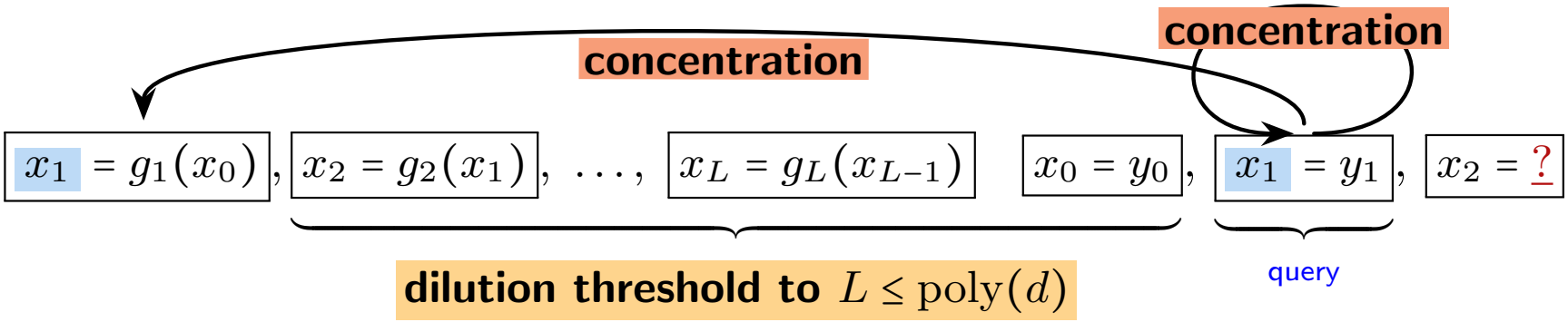
Attention concentration: $\mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{pred},\ell} + \mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{ans},\ell} = 1 - \frac{1}{\mathrm{poly}\, d}$

Length generalization for simple transitive group

Theorem (Short-to-poly length generalization)

If $\mathcal{G}$ is simply transitive, then the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq \mathrm{poly}(d)$.

Mechanism: FFN perfectly captures action; while attention learns to concentrate on the **relevant** clauses.



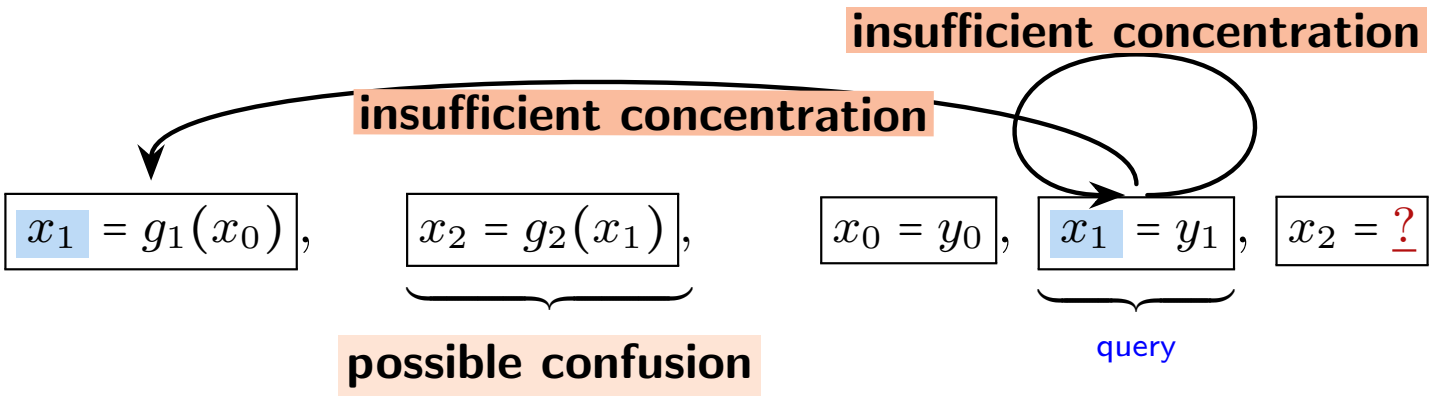
Attention concentration: $\mathbf{Attn}_{\text{ans},\ell\to\text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell\to\text{ans},\ell} = 1 - \frac{1}{\mathrm{poly}\, d}$

Length generalization for symmetry group

Hardness from symmetry $|\mathcal{G}_{y \to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y \to y'}| = 1$.

Length generalization for symmetry group

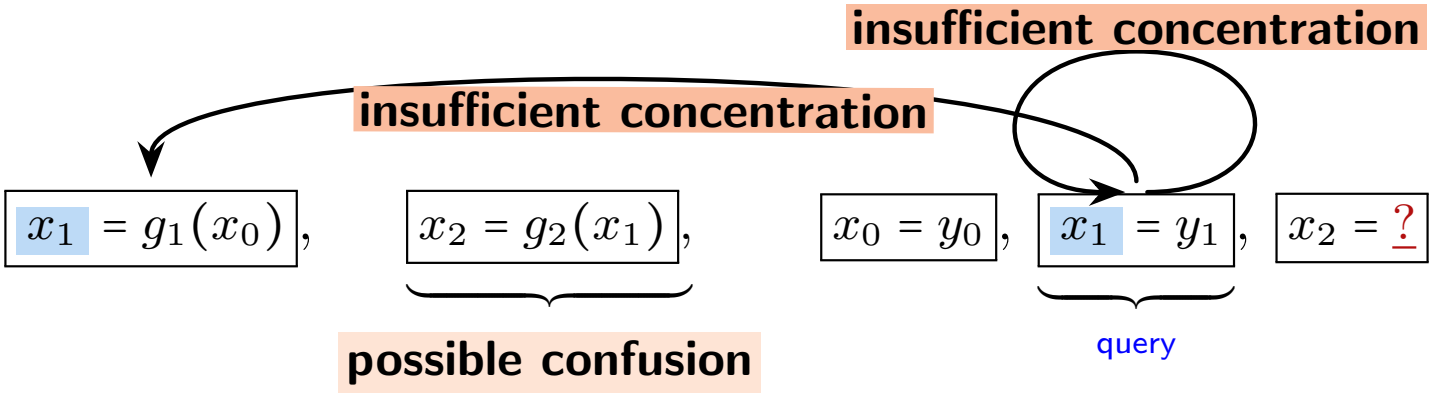
Hardness from symmetry $|\mathcal{G}_{y\to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y\to y'}| = 1$.
Predicate lookup faces many **near-duplicates**:



$$\mathbf{Attn}_{\text{ans},\ell\to\text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell\to\text{ans},\ell} = 1 - \text{small constant}$$

Length generalization for symmetry group

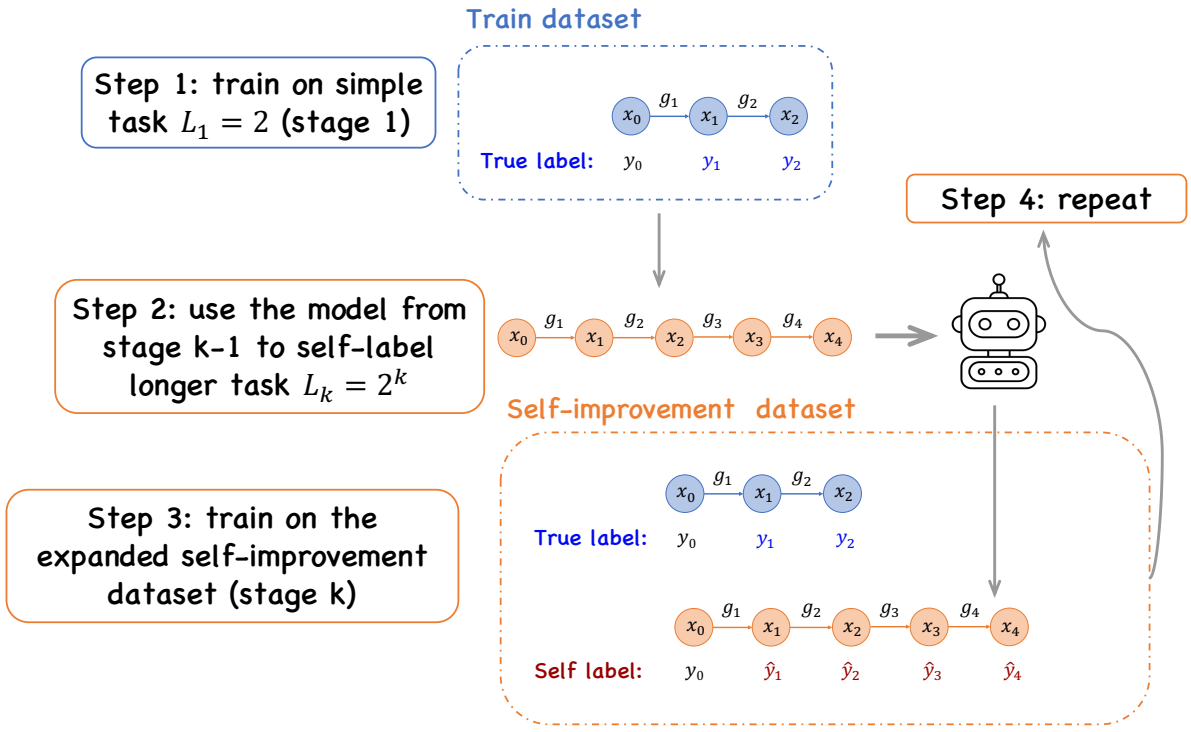
Hardness from symmetry $|\mathcal{G}_{y \to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y \to y'}| = 1$.
Predicate lookup faces many **near-duplicates**:



$$\mathbf{Attn}_{\text{ans},\ell \to \text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell \to \text{ans},\ell} = 1 - \text{small constant}$$

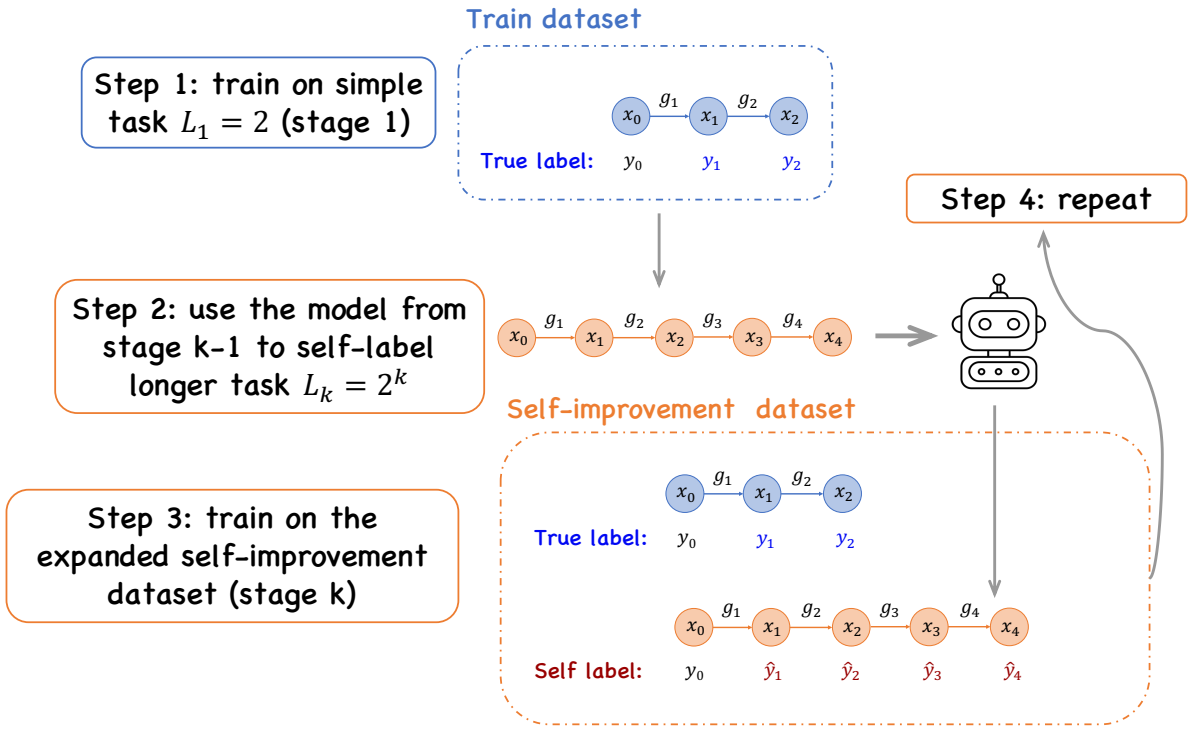
No guarantee beyond **constant**-factor length generalization!

Self-improvement provably helps



Inspired by (Lee et al., 2025)

Self-improvement provably helps

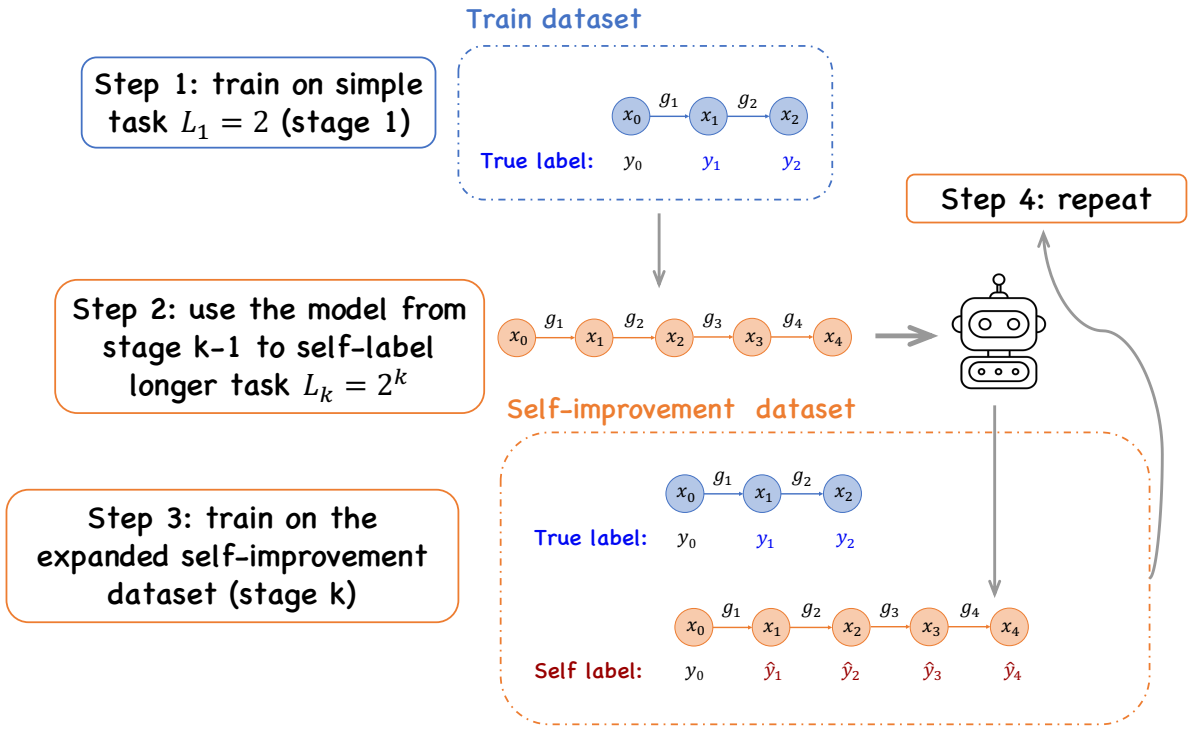


Inspired by (Lee et al., 2025)

Theorem (Self-improvement for length generalization)

If $\mathcal{G}$ is symmetry (S_n), then the model trained via self-training on $L = 1, 2, \cdots, 2^{k-1}$ can generalize to solve task of $L = 2^k$. After $\Theta(\log d)$ stages, it can solve tasks of $L \leq d$.

Self-improvement provably helps



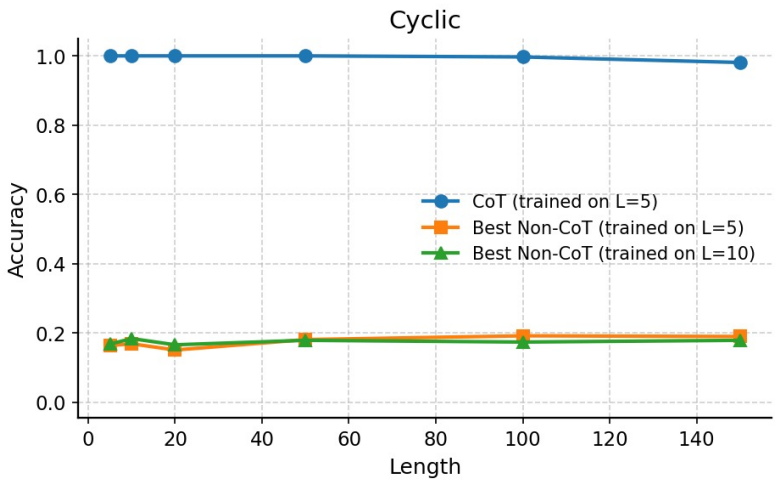
Inspired by (Lee et al., 2025)

Theorem (Self-improvement for length generalization)

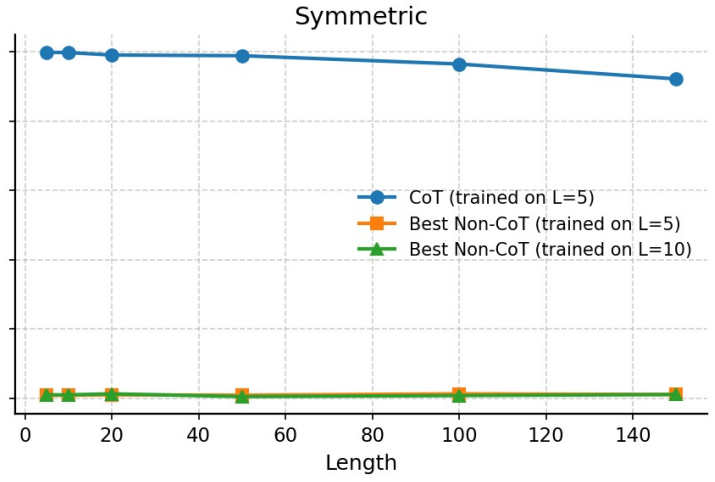
If $\mathcal{G}$ is symmetry (S_n), then the model trained via self-training on $L = 1, 2, \cdots, 2^{k-1}$ can generalize to solve task of $L = 2^k$. After $\Theta(\log d)$ stages, it can solve tasks of $L \leq d$.

Transformer with CoTs **learns** to solve problems in NC^1!

Numerical experiments



Cyclic group (C_6)

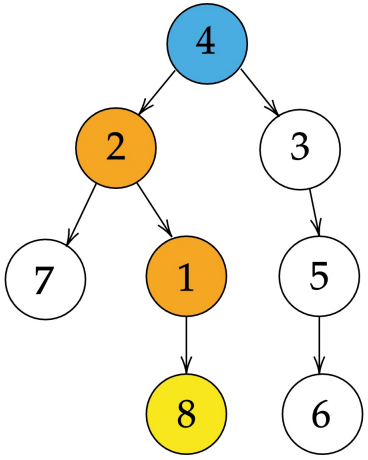


Symmetric group (S_5)

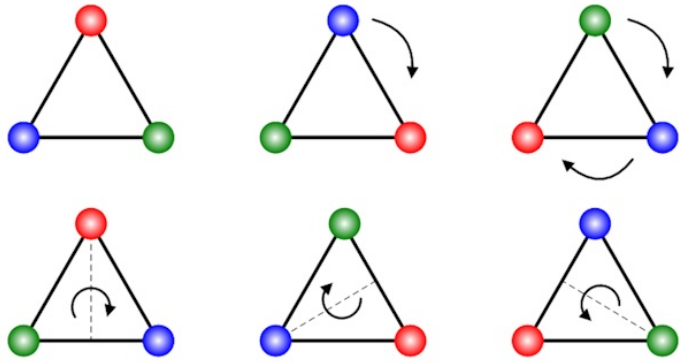
Transformer with CoTs **generalizes** much better than non-CoT training

Summary

symbolic reasoning



reasoning with group action



Take-away: one-layer transformers with CoT training can reason with length generalization (without positional encoding)

Future work:

- Other reasoning tasks?
- Understanding RL for CoT reasoning.

Thanks!

- Multi-head Transformers Provably Learn Symbolic Multi-step Reasoning via Gradient Descent, NeurIPS 2025, arXiv 2508.08222.
- Transformers Learn Chain-of-Thought Reasoning with Length Generalization, NeurIPS 2025, to be arXived soon.

https://yuejiechi.github.io